Prelucrarea Imaginilor

Prof. Dr. Călin-Adrian POPA

Cursul 3

13 Octombrie 2025

3 Convolutional neural networks

- for image data, each example consists of a two-dimensional *grid of pixels*
- depending on whether we are handling black-and-white or color images, each pixel location might be associated with either one or multiple numerical values, respectively
- one way of dealing with this rich structure is to simply discard each image's spatial structure by flattening them into one-dimensional vectors, and feeding them through a fully-connected MLP, which is deeply unsatisfying
- because these networks are invariant to the order of the features, we could get similar results regardless of whether we preserve an order corresponding to the spatial structure of the pixels, or if we permute the columns of our design matrix before fitting the MLP's parameters
- preferably, we would leverage our prior knowledge that nearby pixels are typically related to each other, to build efficient models for learning from image data

3 Convolutional neural networks

- in this chapter, we introduce *convolutional neural networks* (*CNNs*), a powerful family of neural networks that are designed for precisely this purpose
- CNN-based architectures are now ubiquitous in the field of computer vision, and have become so dominant that hardly anyone today would develop a commercial application or enter a competition related to image recognition, object detection, or semantic segmentation, without building off of this approach

3 Convolutional neural networks

- modern CNNs owe their design to inspirations from biology, group theory, and much experimentation
- in addition to their sample efficiency in achieving accurate models, CNNs tend to be computationally efficient, both because they require fewer parameters than fully-connected architectures, and because convolutions are easy to parallelize across GPU cores
- consequently, CNNs are often applied whenever possible, and increasingly they have emerged as good competitors even on tasks with a one-dimensional sequence structure, such as audio, text, and time series analysis, where recurrent neural networks are conventionally used
- some clever adaptations of CNNs have also brought applications on graph-structured data and in recommender systems

3 Convolutional neural networks

- first, we will walk through the basic operations that comprise the backbone of all convolutional networks
- these include the convolutional layers themselves, details including padding and stride, the pooling layers used to aggregate information across adjacent spatial regions, the use of multiple channels at each layer, and a careful discussion of the structure of modern architectures
- we will discuss the example of LeNet, the first convolutional network successfully deployed, long before the rise of modern deep learning
- then, we will dive into full details of some popular and comparatively recent CNN architectures, whose designs represent most of the techniques commonly used today

3.1 From fully-connected layers to convolutions

- to this day, the models that we have discussed so far remain appropriate options when we are dealing with tabular data
- by tabular, we mean that the data consist of rows corresponding to examples and columns corresponding to features
- with tabular data, we might anticipate that the patterns we seek could involve interactions among the features, but we do not assume any structure *a priori* concerning how the features interact
- sometimes, we truly lack knowledge to guide the construction of better architectures; in these cases, an MLP may be the best that we can do
- however, for high-dimensional perceptual data, such structure-less networks may not be suited

3.1 From fully-connected layers to convolutions

- for instance, suppose we want to distinguish cats from dogs
- say that we do a thorough job in data collection, collecting an annotated dataset of one-megapixel photographs
- this means that each input to the network has one million dimensions
- according to our discussions of parameterization cost of fully-connected layers, even an aggressive reduction to one thousand hidden dimensions would require a fully-connected layer characterized by $10^6 \times 10^3 = 10^9$ parameters
- learning the parameters of this network may turn out to be infeasible

3.1 From fully-connected layers to convolutions

- we might object to this argument on the basis that one megapixel resolution may not be necessary
- however, while we might be able to get away with one hundred thousand pixels, our hidden layer of size 1000 grossly underestimates the number of hidden units that it takes to learn good representations of images, so a practical system will still require billions of parameters
- moreover, learning a classifier by fitting so many parameters might require collecting an enormous dataset
- and yet, today, both humans and computers are able to distinguish cats from dogs quite well, seemingly contradicting these intuitions
- that is because images exhibit rich structure that can be exploited by humans and machine learning models alike
- convolutional neural networks (CNNs) are one creative way that machine learning has embraced for exploiting some of the known structure in natural images

3.1 From fully-connected layers to convolutions

- imagine that we want to detect an object in an image
- it seems reasonable that whatever method we use to recognize objects should not be overly concerned with the precise location of the object in the image; ideally, our system should exploit this knowledge
- we can draw some inspiration here from the children's game "Where's Waldo" (depicted in Figure 1)
- the game consists of a number of chaotic scenes with many activities

3.1 From fully-connected layers to convolutions

- Waldo shows up somewhere in each, typically in some unlikely location; the player's goal is to locate him
- despite his characteristic outfit, this can be surprisingly difficult, due to the large number of distractions
- however, *what Waldo looks like* does not depend upon *where Waldo is located*
- we could sweep the image with a Waldo detector that could assign a score to each patch, indicating the likelihood that the patch contains Waldo
- CNNs systematize this idea of *spatial invariance*, exploiting it to learn useful representations with fewer parameters

3.1 From fully-connected layers to convolutions

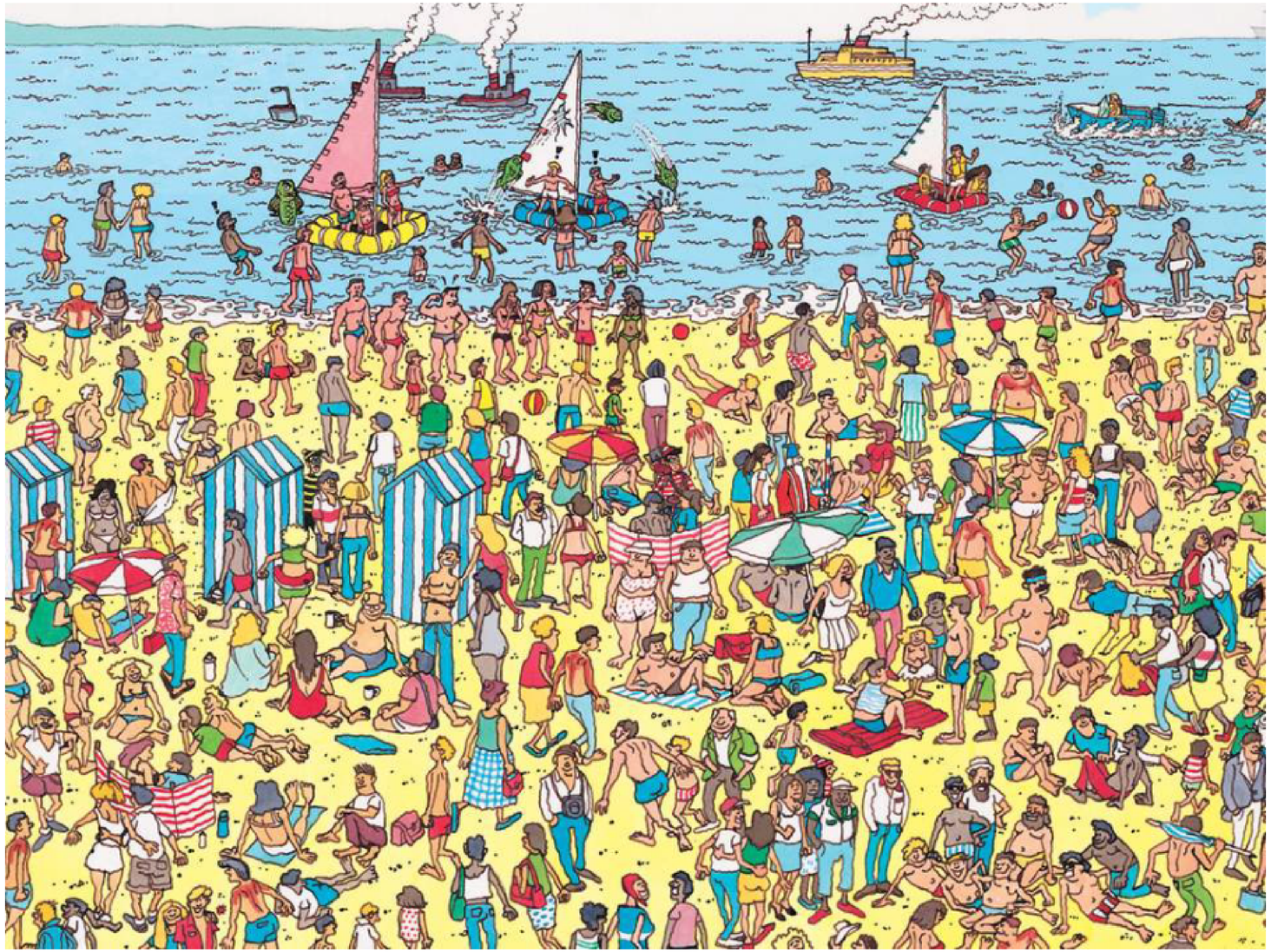

Figure 1: An image of the “Where’s Waldo” game.

3.1 From fully-connected layers to convolutions

- we can now make these intuitions more concrete by enumerating a few desiderata to guide our design of a neural network architecture suitable for computer vision:
  1. In the earliest layers, our network should respond similarly to the same patch, regardless of where it appears in the image. This principle is called *translation invariance*.
  2. The earliest layers of the network should focus on local regions, without regard for the contents of the image in distant regions. This is the *locality* principle. Eventually, these local representations can be *aggregated* to make predictions at the whole image level.
- let us see how this translates into mathematics; first, we need to discuss the notion of tensors
- just as vectors generalize scalars, and matrices generalize vectors, we can build data structures with even more axes
- *tensors* give us a generic way of describing *n*-dimensional arrays with an arbitrary number of axes
- vectors, for example, are first-order tensors, and matrices are second-order tensors
- tensors are denoted with plain capital letters (e.g., X, Y, and Z) and their indexing mechanism (e.g., x_{ijk} and $[\mathsf{X}]_{i,j,k}$) is similar to that of matrices

3.1 From fully-connected layers to convolutions

- to start off, we can consider an MLP with two-dimensional images $\mathbf{X}$ as inputs and their immediate hidden representations $\mathbf{H}$ similarly represented as matrices, where both $\mathbf{X}$ and $\mathbf{H}$ have the same shape
- we now conceive of not only the inputs, but also the hidden representations as possessing spatial structure
- let $[\mathbf{X}]_{i,j}$ and $[\mathbf{H}]_{i,j}$ denote the pixel at location (i, j) in the input image and hidden representation, respectively
- consequently, to have each of the hidden units receive input from each of the input pixels, we would switch from using weight matrices (as we did previously in MLPs) to representing our parameters as fourth-order weight tensors W

3.1 From fully-connected layers to convolutions

- suppose that **U** contains biases, we could formally express the fully-connected layer as:

$$\begin{aligned}[\mathbf{H}]_{i,j} &= [\mathbf{U}]_{i,j} + \sum_k \sum_l [\mathsf{W}]_{i,j,k,l}[\mathbf{X}]_{k,l} \\ &= [\mathbf{U}]_{i,j} + \sum_a \sum_b [\mathsf{V}]_{i,j,a,b}[\mathbf{X}]_{i+a,j+b},\end{aligned}$$

  where the switch from W to V is entirely cosmetic for now, since there is a one-to-one correspondence between coefficients in both fourth-order tensors
- we simply re-index the subscripts (k, l), such that $k = i + a$ and $l = j + b$; in other words, we set $[\mathsf{V}]_{i,j,a,b} = [\mathsf{W}]_{i,j,k,l}$
- the indices a and b run over both positive and negative offsets, covering the entire image
- for any given location (i, j) in the hidden representation $[\mathbf{H}]_{i,j}$, we compute its value by summing over pixels in **X**, centered around (i, j), and weighted by $[\mathsf{V}]_{i,j,a,b}$

3.1 From fully-connected layers to convolutions

- now, let us invoke the first principle established above: *translation invariance*
- this implies that a shift in the input **X** should simply lead to a shift in the hidden representation **H**
- this is only possible if V and **U** do not actually depend on (i, j), i.e., we have $[\mathrm{V}]_{i,j,a,b} = [\mathbf{V}]_{a,b}$ and **U** is a constant, say u
- as a result, we can simplify the definition for **H**:

$$[\mathbf{H}]_{i,j} = u + \sum_a \sum_b [\mathbf{V}]_{a,b} [\mathbf{X}]_{i+a,j+b}.$$

- this is a *convolution*: we are effectively weighting pixels at $(i + a, j + b)$, in the vicinity of location (i, j), with coefficients $[\mathbf{V}]_{a,b}$ to obtain the value $[\mathbf{H}]_{i,j}$
- note that $[\mathbf{V}]_{a,b}$ needs many fewer coefficients than $[\mathrm{V}]_{i,j,a,b}$, since it no longer depends on the location within the image, which is a significant progress

3.1 From fully-connected layers to convolutions

- now, let us invoke the second principle: *locality*
- as motivated above, we believe that we should not have to look very far away from location (i, j) in order to gather relevant information to assess what is going on at $[\mathbf{H}]_{i,j}$
- this means that, outside some range $|a| > \Delta$ or $|b| > \Delta$, we should set $[\mathbf{V}]_{a,b} = 0$
- equivalently, we can rewrite $[\mathbf{H}]_{i,j}$ as:

$$[\mathbf{H}]_{i,j} = u + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} [\mathbf{V}]_{a,b} [\mathbf{X}]_{i+a,j+b}. \tag{1}$$

3.1 From fully-connected layers to convolutions

- note that (1), in a nutshell, is a *convolutional layer*
- *convolutional neural networks* (*CNNs*) are a special family of neural networks that contain convolutional layers
- in the deep learning research community, **V** is referred to as a *convolution kernel*, a *filter*, or simply the layer's *weights*, which are often learnable parameters
- when the local region is small, the difference as compared with a fully-connected network can be dramatic
- while, previously, we might have required billions of parameters to represent just a single layer in an image-processing network, we now typically need just a few hundred, without altering the dimensionality of either the inputs or the hidden representations
- the price paid for this drastic reduction in parameters is that our features are now translation invariant and that our layer can only incorporate local information, when determining the value of each hidden activation

3.1 From fully-connected layers to convolutions

- before going further, we should briefly review why the above operation is called a *convolution*
- in mathematics, the convolution between two functions, say $f, g : \mathbb{R}^d \to \mathbb{R}$ is defined as:

$$(f * g)(\boldsymbol{x}) = \int f(\boldsymbol{z}) g(\boldsymbol{x} - \boldsymbol{z}) d\boldsymbol{z}.$$

- that is, we measure the *overlap* between f and g when one function is "flipped" and shifted by $\boldsymbol{x}$
- whenever we have discrete objects, the integral turns into a sum
- for instance, for vectors from the set of square summable infinite dimensional vectors with index running over $\mathbb{Z}$, we obtain the following definition:

$$(f * g)(i) = \sum_a f(a) g(i - a).$$

3.1 From fully-connected layers to convolutions

- for two-dimensional tensors, we have a corresponding sum with indices (a, b) for f and $(i - a, j - b)$ for g, respectively:

$$(f * g)(i, j) = \sum_a \sum_b f(a, b) g(i - a, j - b). \tag{2}$$

- this looks similar to (1), with one major difference
- rather than using $(i + a, j + b)$, we are using the difference instead
- note, though, that this distinction is mostly cosmetic, since we can always match the notation between (1) and (2)
- our original definition in (1) more properly describes a *cross-correlation*; we will come back to this in the following section

3.1 From fully-connected layers to convolutions

- returning to our Waldo detector, let us see what this looks like
- the convolutional layer picks windows of a given size and weighs intensities according to the filter **V**, as demonstrated in Figure 2
- we might aim to learn a model so that wherever the “Waldoness” is highest, we should find a peak in the hidden layer representations

3.1 From fully-connected layers to convolutions

Figure 2: Detecting Waldo.

3.1 From fully-connected layers to convolutions

- there is just one problem with this approach; so far, we ignored that images consist of 3 channels: red, green, and blue
- in reality, images are not two-dimensional objects but rather third-order tensors, characterized by a height, width, and channel, e.g., with shape $1024 \times 1024 \times 3$ pixels
- while the first two of these axes concern spatial relationships, the third can be regarded as assigning a multidimensional representation to each pixel location
- we thus index X as $[\mathsf{X}]_{i,j,k}$
- the convolutional filter has to adapt accordingly; instead of $[\mathbf{V}]_{a,b}$, we now have $[\mathsf{V}]_{a,b,c}$

3.1 From fully-connected layers to convolutions

- moreover, just as our input consists of a third-order tensor, it turns out to be a good idea to similarly formulate our hidden representations as third-order tensors H
- in other words, rather than just having a single hidden representation corresponding to each spatial location, we want an *entire vector* of *hidden representations* corresponding to each spatial location
- we could think of the hidden representations as comprising a number of two-dimensional grids, stacked on top of each other
- as in the inputs, these are sometimes called *channels*
- they are also sometimes called *feature maps*, as each provides a spatialized set of learned features to the subsequent layer
- intuitively, we might imagine that, at lower layers that are closer to inputs, some channels could become specialized to recognize edges, while others could recognize textures

3.1 From fully-connected layers to convolutions

- to support multiple channels in both inputs (X) and hidden representations (H), we can add a fourth coordinate to V: $[\mathrm{V}]_{a,b,c,d}$
- correspondingly, the bias u becomes a vector $\mathbf{u}$
- putting everything together, we have:

$$[\mathrm{H}]_{i,j,d} = [\mathbf{u}]_d + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} \sum_{c} [\mathrm{V}]_{a,b,c,d} [\mathrm{X}]_{i+a,j+b,c}, \tag{3}$$

  where d indexes the *output channels* in the hidden representations H
- the subsequent convolutional layer will go on to take a third-order tensor, H, as the input
- being more general, (3) is the definition of a *convolutional layer* for *multiple channels*, where V is a kernel or filter of the layer

3.1 From fully-connected layers to convolutions

- there are still many operations that we need to address
- for instance, we need to figure out how to combine all the hidden representations to a single output, e.g., whether there is a Waldo *anywhere* in the image
- we also need to decide how to compute things efficiently, how to combine multiple layers, appropriate activation functions, and how to make reasonable design choices to yield networks that are effective in practice
- we turn to these issues in the next sections

3.2 Convolutions for images

- now that we understand how convolutional layers work in theory, we are ready to see how they work in practice
- building on our motivation of convolutional neural networks as efficient architectures for exploring structure in image data, we stick with images as our running example
- recall that, strictly speaking, convolutional layers are a misnomer, since the operations they express are more accurately described as cross-correlations
- based on our descriptions of convolutional layers, in such a layer, an input tensor and a kernel tensor are combined to produce an output tensor through a cross-correlation operation

3.2 Convolutions for images

- let us ignore channels for now, and see how this works with two-dimensional data and hidden representations
- in Figure 3, the input is a two-dimensional tensor with a height of 3 and width of 3
- we mark the shape of the tensor as 3×3 or $(3, 3)$
- the height and width of the kernel are both 2
- the shape of the *kernel window* (or *convolution window*) is given by the height and width of the kernel (here, it is 2×2)

3.2 Convolutions for images

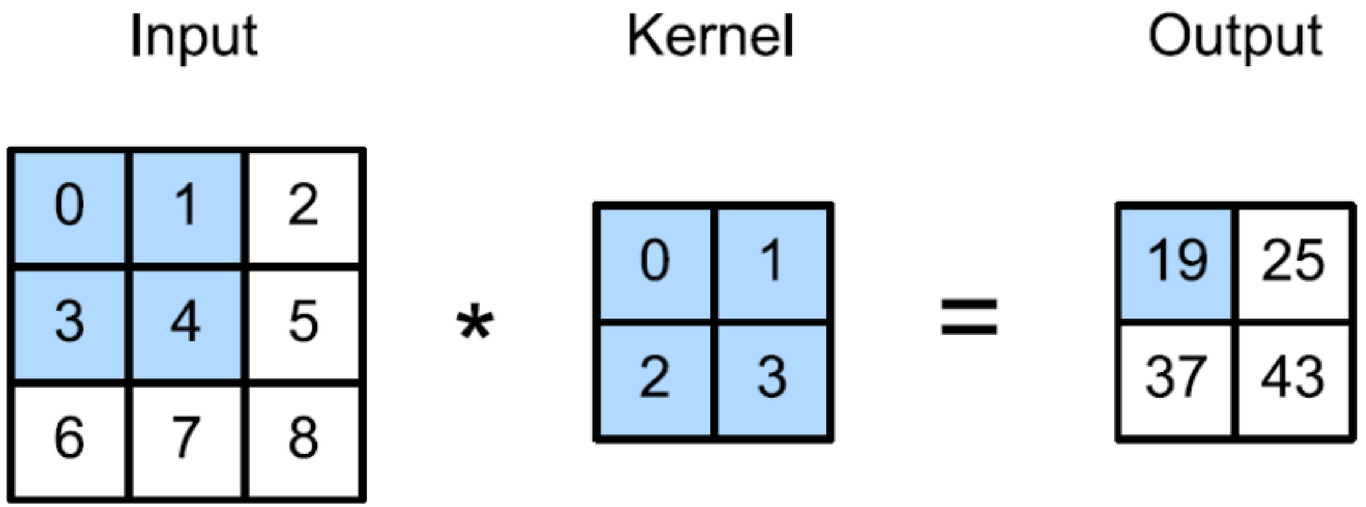

Figure 3: Two-dimensional cross-correlation operation. The shaded portions are the first output element, as well as the input and kernel tensor elements used for the output computation: $0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 = 19$.

3.2 Convolutions for images

- in the two-dimensional cross-correlation operation, we begin with the convolution window positioned at the upper-left corner of the input tensor, and slide it across the input tensor, both from left to right and top to bottom
- when the convolution window slides to a certain position, the input subtensor contained in that window and the kernel tensor are multiplied element-wise, and the resulting tensor is summed up, yielding a single scalar value
- this result gives the value of the output tensor at the corresponding location
- here, the output tensor has a height of 2 and width of 2, and the four elements are derived from the two-dimensional cross-correlation operation:

$$
\begin{aligned}
0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 &= 19, \\
1 \times 0 + 2 \times 1 + 4 \times 2 + 5 \times 3 &= 25, \\
3 \times 0 + 4 \times 1 + 6 \times 2 + 7 \times 3 &= 37, \\
4 \times 0 + 5 \times 1 + 7 \times 2 + 8 \times 3 &= 43.
\end{aligned}
$$

3.2 Convolutions for images

- note that, along each axis, the output size is slightly smaller than the input size
- because the kernel has width and height greater than one, we can only properly compute the cross-correlation for locations where the kernel fits wholly within the image, the output size is given by the input size $n_h \times n_w$ minus the size of the convolution kernel $k_h \times k_w$ via:

$$(n_h - k_h + 1) \times (n_w - k_w + 1).$$

- this is the case, since we need enough space to “shift” the convolution kernel across the image
- later, we will see how to keep the size unchanged by padding the image with zeros around its boundary, so that there is enough space to shift the kernel

3.2 Convolutions for images

- a convolutional layer cross-correlates the input and kernel, and adds a scalar bias to produce an output
- the two parameters of a convolutional layer are the *kernel* and the scalar *bias*
- when training models based on convolutional layers, we typically initialize the kernels randomly, just as we would with a fully-connected layer
- in $h \times w$ convolution or a $h \times w$ convolution kernel, the height and width of the convolution kernel are h and w, respectively
- we also refer to a convolutional layer with a $h \times w$ convolution kernel simply as a $h \times w$ convolutional layer

3.2 Convolutions for images

- let us take a moment to discuss a simple application of a convolutional layer: detecting the edge of an object in an image by finding the location of the pixel change
- in this case, the kernel can be constructed as $[1, -1]$, i.e., it has a height of 1 and a width of 2
- designing an edge detector by finite differences $[1, -1]$ is good, if we know this is precisely what we are looking for
- however, as we look at larger kernels, and consider successive layers of convolutions, it might be impossible to specify precisely what each filter should be doing, manually
- thus, we need to *learn* these kernels

3.2 Convolutions for images

- recall our observation of the correspondence between the cross-correlation and convolution operations
- here, let us continue to consider two-dimensional convolutional layers
- what if such layers perform strict convolution operations as defined in (2), instead of cross-correlations?
- in order to obtain the output of the strict *convolution* operation, we only need to flip the two-dimensional kernel tensor both horizontally and vertically, and then perform the *cross-correlation* operation with the input tensor
- it is noteworthy that, since kernels are learned from data in deep learning, the outputs of convolutional layers remain unaffected, no matter if such layers perform either the strict convolution operations, or the cross-correlation operations

3.2 Convolutions for images

- to illustrate this, suppose that a convolutional layer performs *cross-correlation* and learns the kernel in Figure 3, which is denoted as the matrix **K** here
- assuming that other conditions remain unchanged, when this layer performs strict *convolution* instead, the learned kernel $\mathbf{K}'$ will be the same as **K** after $\mathbf{K}'$ is flipped both horizontally and vertically
- that is to say, when the convolutional layer performs strict *convolution* for the input in Figure 3 and $\mathbf{K}'$, the same output in Figure 3 (cross-correlation of the input and **K**) will be obtained
- in keeping with standard terminology with deep learning literature, we will continue to refer to the cross-correlation operation as a convolution even though, strictly-speaking, it is slightly different
- besides, we use the term *element* to refer to an entry (or component) of any tensor representing a layer representation or a convolution kernel

3.2 Convolutions for images

- as described before, the convolutional layer output in Figure 3 is sometimes called a *feature map*, as it can be regarded as the learned representations (features) in the spatial dimensions (e.g., width and height) to the subsequent layer
- in CNNs, for any element x of some layer, its *receptive field* refers to all the elements (from all the previous layers) that may affect the calculation of x during the forward propagation
- note that the receptive field may be larger than the actual size of the input

3.2 Convolutions for images

- let us continue to use Figure 3 to explain the receptive field
- given the 2×2 convolution kernel, the *receptive field* of the shaded output element (of value 19) is the four elements in the shaded portion of the input
- now, let us denote the 2×2 output as **Y** and consider a deeper CNN with an additional 2×2 convolutional layer that takes **Y** as its input, outputting a single element z
- in this case, the receptive field of z on **Y** includes all the four elements of **Y**, while the receptive field on the input includes all the nine input elements
- thus, when any element in a feature map needs a larger receptive field to detect input features over a broader area, we can build a deeper network

3.3 Padding and stride

- in the previous example of Figure 3, our input had both a height and width of 3 and our convolution kernel had both a height and width of 2, yielding an output representation with dimension 2×2
- as we generalized in Section 3.2, assuming that the input shape is $n_h \times n_w$ and the convolution kernel shape is $k_h \times k_w$, then the output shape will be $(n_h - k_h + 1) \times (n_w - k_w + 1)$
- therefore, the output shape of the convolutional layer is determined by the shape of the input and the shape of the convolution kernel

3.3 Padding and stride

- in several cases, we incorporate techniques, including *padding* and *strided convolutions*, that affect the size of the output
- as motivation, note that, since kernels generally have width and height greater than 1, after applying many successive convolutions, we tend to wind up with outputs that are considerably smaller than our input
- if we start with a 240×240 pixel image, 10 layers of 5×5 convolutions reduce the image to 200×200 pixels, slicing off 30% of the image, and with it ignoring any interesting information on the boundaries of the original image
- *padding* is the most popular tool for handling this issue
- in other cases, we may want to reduce the dimensionality drastically, e.g., if we find the original input resolution to be too big
- *strided convolutions* are a popular technique that can help in these instances

3.3 Padding and stride

- as described above, one tricky issue when applying convolutional layers is that we tend to lose pixels on the perimeter of our image
- since we typically use small kernels, for any given convolution, we might only lose a few pixels, but this can add up as we apply many successive convolutional layers
- one straightforward solution to this problem is to add extra pixels around the boundary of our input image, thus increasing the effective size of the image
- typically, we set the values of the extra pixels to zero
- in Figure 4, we pad a 3×3 input, increasing its size to 5×5; the corresponding output then increases to a 4×4 matrix
- the shaded portions are the first output element, as well as the input and kernel tensor elements used for the output computation: $0 \times 0 + 0 \times 1 + 0 \times 2 + 0 \times 3 = 0$

3.3 Padding and stride

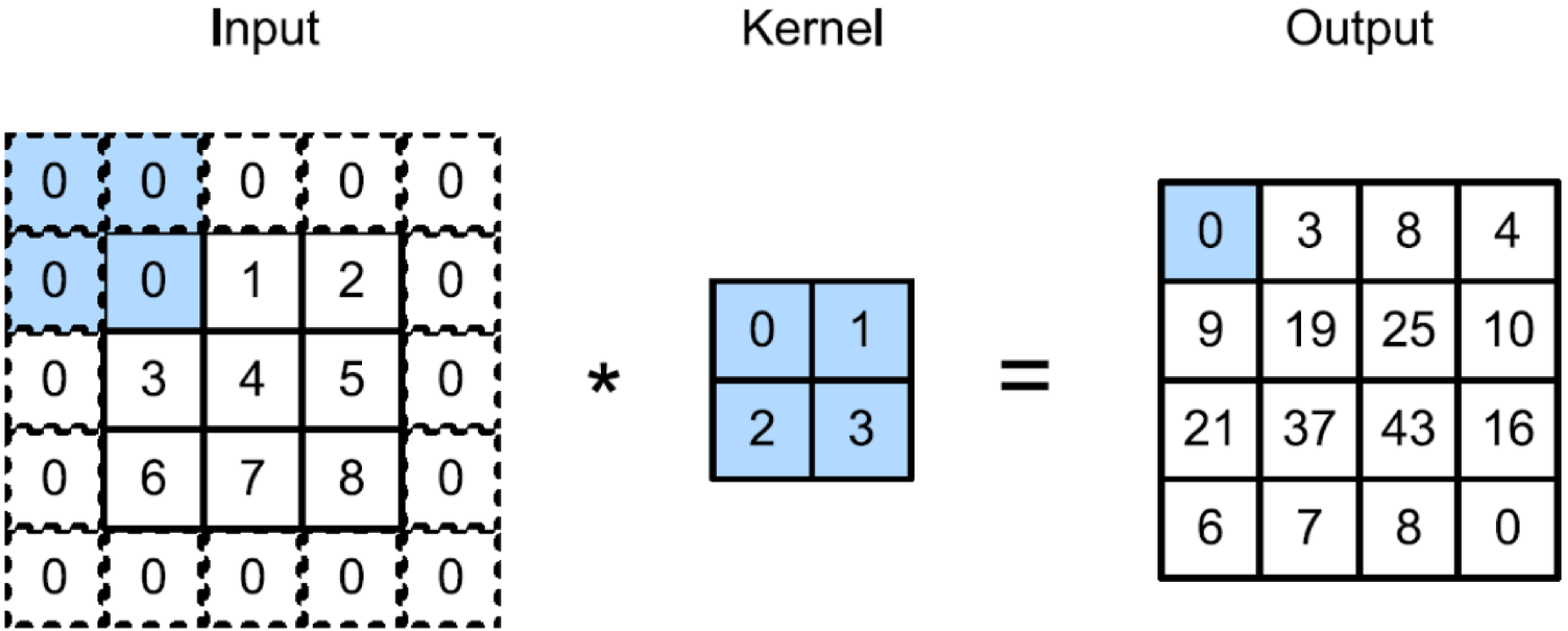

Figure 4: Two-dimensional cross-correlation with padding.

3.3 Padding and stride

- in general, if we add a total of p_h rows of padding (roughly half on top and half on bottom) and a total of p_w columns of padding (roughly half on the left and half on the right), the output shape will be:

$$(n_h - k_h + p_h + 1) \times (n_w - k_w + p_w + 1).$$

- this means that the height and width of the output will increase by p_h and p_w, respectively
- in many cases, we will want to set $p_h = k_h - 1$ and $p_w = k_w - 1$ to give the input and output the same height and width
- this will make it easier to predict the output shape of each layer, when constructing the network
- assuming that k_h is odd here, we will pad $p_h/2$ rows on both sides of the height
- if k_h is even, one possibility is to pad $\lceil p_h/2 \rceil$ rows on the top of the input and $\lfloor p_h/2 \rfloor$ rows on the bottom
- we will proceed in the same way for the width

3.3 Padding and stride

- CNNs commonly use convolution kernels with odd height and width values, such as 1, 3, 5, or 7
- choosing odd kernel sizes has the benefit that we can preserve the spatial dimensionality while padding with the same number of rows on top and bottom, and the same number of columns on left and right
- moreover, this practice of using odd kernels and padding to precisely preserve dimensionality offers a clear benefit
- for any two-dimensional tensor $\mathbf{X}$, when the kernel's size is odd and the number of padding rows and columns on all sides are the same, producing an output with the same height and width as the input, we know that the output $[\mathbf{Y}]_{i,j}$ is calculated by cross-correlation of the input and convolution kernel with the window centered on $[\mathbf{X}]_{i,j}$

3.3 Padding and stride

- when computing the cross-correlation, we start with the convolution window at the upper-left corner of the input tensor, and then slide it over all locations both down and to the right
- in previous examples, we default to sliding one element at a time
- however, sometimes, either for computational efficiency, or because we wish to downsample, we move our window more than one element at a time, skipping the intermediate locations
- we refer to the number of rows and columns traversed per slide as the *stride*
- so far, we have used strides of 1, both for height and width; sometimes, we may want to use a larger stride

3.3 Padding and stride

- Figure 5 shows a two-dimensional cross-correlation operation with a stride of 3 vertically and 2 horizontally
- the shaded portions are the output elements, as well as the input and kernel tensor elements used for the output computation: $0 \times 0 + 0 \times 1 + 1 \times 2 + 2 \times 3 = 8$, $0 \times 0 + 6 \times 1 + 0 \times 2 + 0 \times 3 = 6$
- we can see that, when the second element of the first column is outputted, the convolution window slides down three rows
- the convolution window slides two columns to the right when the second element of the first row is outputted
- when the convolution window continues to slide two columns to the right on the input, there is no output, because the input element cannot fill the window (unless we add another column of padding)

3.3 Padding and stride

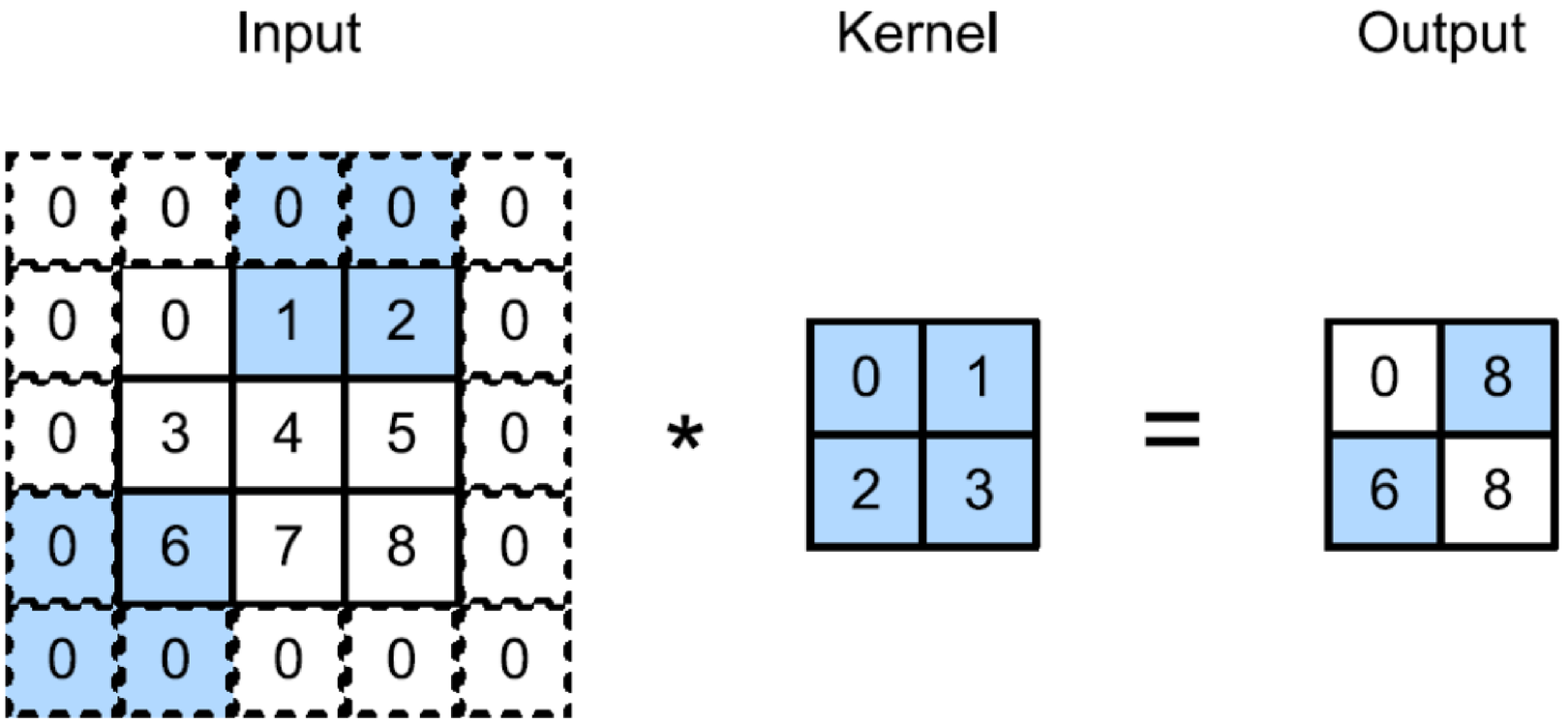

Figure 5: Cross-correlation with strides of 3 and 2 for height and width, respectively.

3.3 Padding and stride

- in general, when the stride for the height is s_h and the stride for the width is s_w, the output shape is:

$$\lfloor (n_h - k_h + p_h + s_h)/s_h \rfloor \times \lfloor (n_w - k_w + p_w + s_w)/s_w \rfloor.$$

- if we set $p_h = k_h - 1$ and $p_w = k_w - 1$, then the output shape will be simplified to $\lfloor (n_h + s_h - 1)/s_h \rfloor \times \lfloor (n_w + s_w - 1)/s_w \rfloor$
- going a step further, if the input height and width are divisible by the strides on the height and width, then the output shape will be $(n_h/s_h) \times (n_w/s_w)$

3.3 Padding and stride

- for the sake of brevity, when the padding number on both sides of the input height and width are p_h and p_w respectively, we call the padding (p_h, p_w)
- specifically, when $p_h = p_w = p$, the padding is p
- when the strides on the height and width are s_h and s_w, respectively, we call the stride (s_h, s_w)
- specifically, when $s_h = s_w = s$, the stride is s
- by default, the padding is 0 and the stride is 1
- in practice, we rarely use inhomogeneous strides or padding, i.e., we usually have $p_h = p_w$ and $s_h = s_w$

3.4 Multiple input and multiple output channels

- while we have described the multiple channels that comprise each image (e.g., color images have the standard RGB channels to indicate the amount of red, green, and blue) and convolutional layers for multiple channels in Section 3.1, until now, we simplified all of our numerical examples by working with just a single input and a single output channel
- this has allowed us to think of our inputs, convolution kernels, and outputs each as two-dimensional tensors
- when we add channels into the mix, our inputs and hidden representations both become three-dimensional tensors; for example, each RGB input image has shape $3 \times h \times w$
- we refer to this axis, with a size of 3, as the *channel dimension*
- in this section, we will take a deeper look at convolution kernels with multiple input and multiple output channels

3.4 Multiple input and multiple output channels

- when the input data contain multiple channels, we need to construct a convolution kernel with the same number of input channels as the input data, so that it can perform cross-correlation with the input data
- assuming that the number of channels for the input data is c_{in}, the number of input channels of the convolution kernel also needs to be c_{in}
- if our convolution kernel's window shape is $k_h \times k_w$, then, when $c_{in} = 1$, we can think of our convolution kernel as just a two-dimensional tensor of shape $k_h \times k_w$
- however, when $c_{in} > 1$, we need a kernel that contains a tensor of shape $k_h \times k_w$ for every input channel
- concatenating these c_{in} tensors together yields a convolution kernel of shape $c_{in} \times k_h \times k_w$

3.4 Multiple input and multiple output channels

- since the input and convolution kernel each have c_{in} channels, we can perform a cross-correlation operation on the two-dimensional tensor of the input and the two-dimensional tensor of the convolution kernel for each channel, adding the c_{in} results together (summing over the channels) to yield a two-dimensional tensor
- this is the result of a two-dimensional cross-correlation between a multi-channel input and a multi-input-channel convolution kernel
- in Figure 6, we demonstrate an example of a two-dimensional cross-correlation with two input channels
- the shaded portions are the first output element, as well as the input and kernel tensor elements used for the output computation:
$(1 \times 1 + 2 \times 2 + 4 \times 3 + 5 \times 4) + (0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3) = 56.$

3.4 Multiple input and multiple output channels

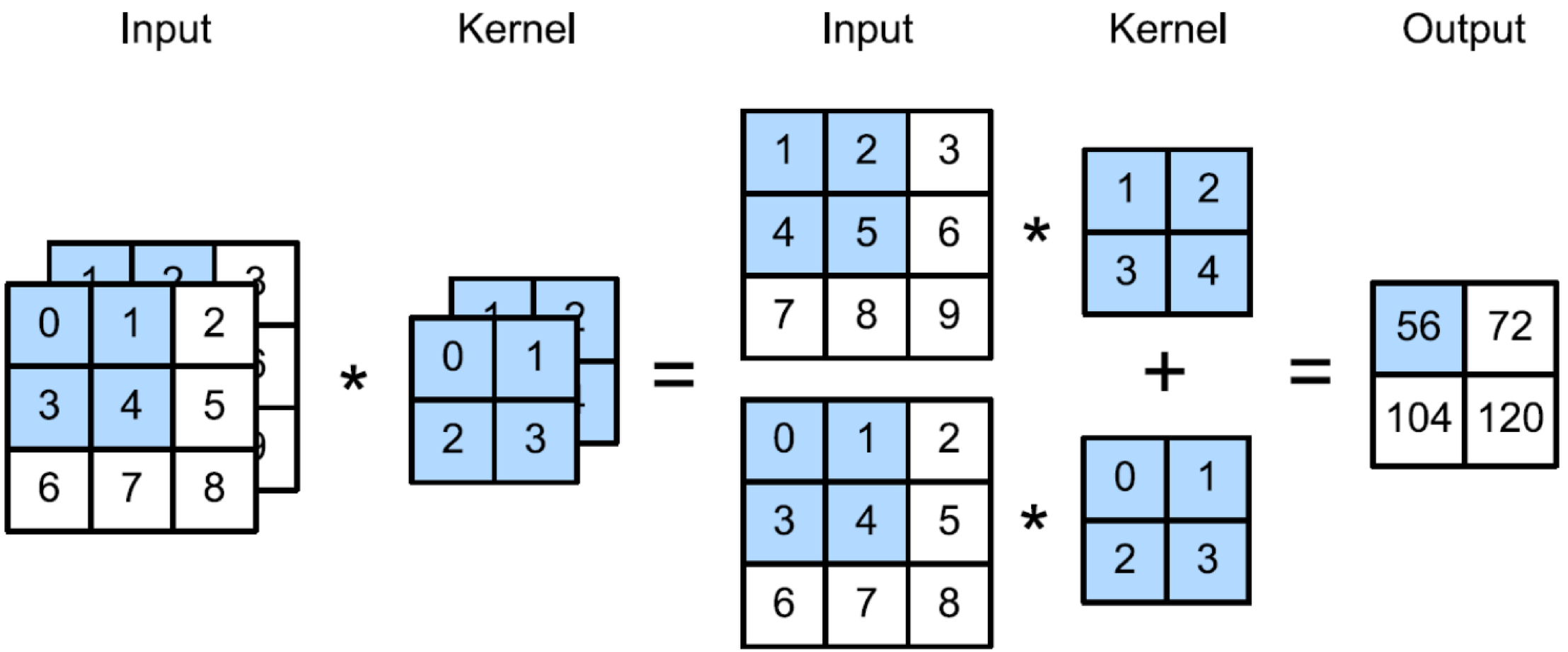

Figure 6: Cross-correlation computation with 2 input channels.

3.4 Multiple input and multiple output channels

- regardless of the number of input channels, so far we always ended up with one output channel
- however, as we discussed in Section 3.1, it turns out to be essential to have multiple channels at each layer
- in the most popular neural network architectures, we actually increase the channel dimension as we go higher up in the neural network, typically downsampling to trade off spatial resolution for greater *channel depth*
- intuitively, we could think of each channel as responding to some different set of features
- reality is a bit more complicated than the most naive interpretations of this intuition, since representations are not learned independently, but are rather optimized to be jointly useful
- so it may not be that a single channel learns an edge detector, but rather that some direction in channel space corresponds to detecting edges

3.4 Multiple input and multiple output channels

- denote by c_{in} and c_{out} the number of input and output channels, respectively, and let k_h and k_w be the height and width of the kernel
- to get an output with multiple channels, we can create a kernel tensor of shape $c_{in} \times k_h \times k_w$ for *every* output channel
- we concatenate them on the output channel dimension, so that the shape of the convolution kernel is $c_{out} \times c_{in} \times k_h \times k_w$
- in cross-correlation operations, the result on each output channel is calculated from the convolution kernel corresponding to that output channel, and takes input from all channels in the input tensor

3.4 Multiple input and multiple output channels

- at first, a 1×1 convolution, i.e., $k_h = k_w = 1$, does not seem to make much sense
- after all, a convolution correlates adjacent pixels; a 1×1 convolution obviously does not
- nonetheless, they are popular operations that are sometimes included in the designs of complex deep networks
- because the minimum window is used, the 1×1 convolution loses the ability of larger convolutional layers to recognize patterns consisting of interactions among adjacent elements in the height and width dimensions
- the only computation of the 1×1 convolution occurs on the channel dimension

3.4 Multiple input and multiple output channels

- Figure 7 shows the cross-correlation computation using the 1×1 convolution kernel with 3 input channels and 2 output channels
- note that the inputs and outputs have the same height and width
- each element in the output is derived from a linear combination of elements *at the same position* in the input image
- we could think of the 1×1 convolutional layer as constituting a fully-connected layer applied at every single pixel location to transform the c_{in} corresponding input values into c_{out} output values
- because this is still a convolutional layer, the weights are tied across pixel location
- thus, the 1×1 convolutional layer requires $c_{out} \times c_{in}$ weights (plus the bias)

3.4 Multiple input and multiple output channels

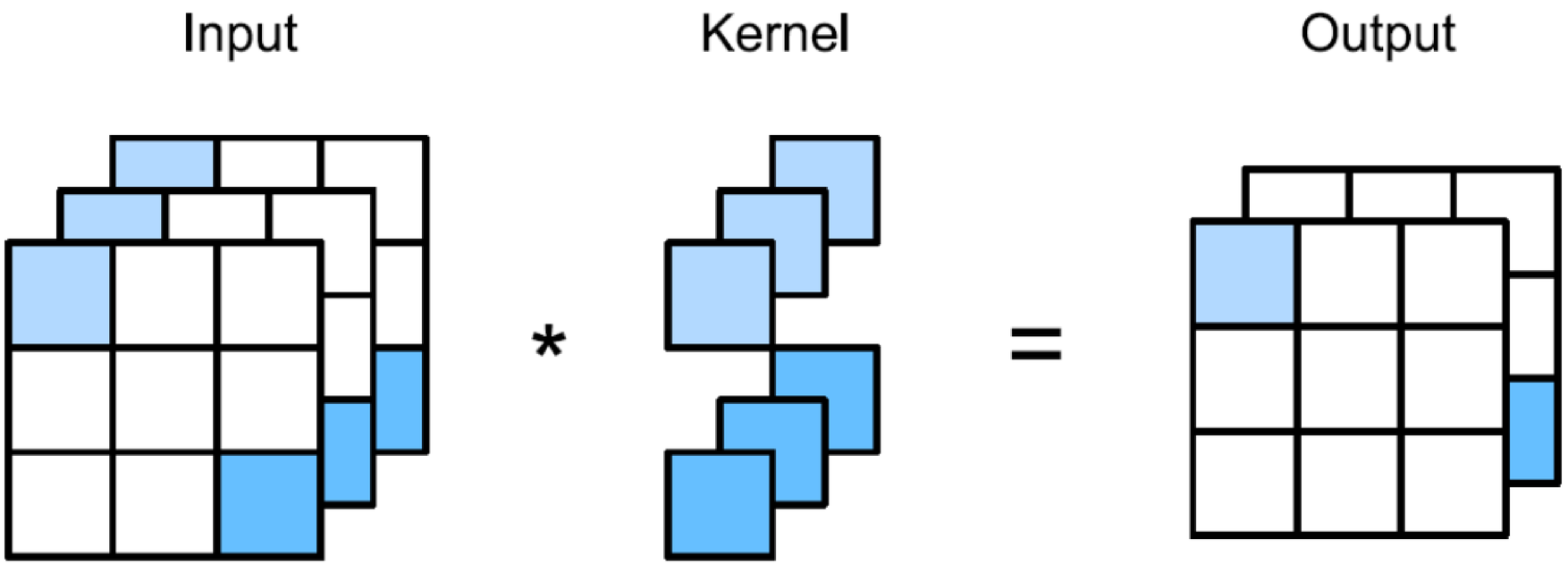

Figure 7: The cross-correlation computation uses the 1×1 convolution kernel with 3 input channels and 2 output channels. The input and output have the same height and width.

3.5 Pooling

- often, as we process images, we want to gradually reduce the spatial resolution of our hidden representations, aggregating information so that the higher up we go in the network, the larger the receptive field (in the input) to which each hidden node is sensitive
- often, our ultimate task asks some global question about the image, e.g., *does it contain a cat*?
- so, typically, the units of our final layer should be sensitive to the entire input
- by gradually aggregating information, yielding coarser and coarser maps, we accomplish this goal of ultimately learning a global representation, while keeping all of the advantages of convolutional layers at the intermediate layers of processing

3.5 Pooling

- moreover, when detecting lower-level features, such as edges, we often want our representations to be somewhat invariant to translation
- for instance, if we take the image $\mathbf{X}$, with a sharp delineation between black and white, and shift the whole image by one pixel to the right, i.e., $[\mathbf{Z}]_{i,j} = [\mathbf{X}]_{i,j+1}$, then the output for the new image $\mathbf{Z}$ might be vastly different
- the edge will have shifted by one pixel; in reality, objects hardly ever occur exactly at the same place
- in fact, even with a tripod and a stationary object, vibration of the camera due to the movement of the shutter might shift everything by a pixel or so (high-end cameras are loaded with special features to address this problem)

3.5 Pooling

- this section introduces *pooling layers*, which serve the dual purposes of mitigating the sensitivity of convolutional layers to location and of spatially downsampling representations
- like convolutional layers, *pooling* operators consist of a fixed-shape window that is slid over all regions in the input according to its stride, computing a single output for each location traversed by the fixed-shape window (sometimes known as the *pooling window*)
- however, unlike the cross-correlation computation of the inputs and kernels in the convolutional layer, the pooling layer contains no parameters (there is no *kernel*)

3.5 Pooling

- instead, pooling operators are deterministic, typically calculating either the maximum or the average value of the elements in the pooling window
- these operations are called *maximum pooling* (*max-pooling*, for short) and *average pooling*, respectively
- in both cases, as with the cross-correlation operator, we can think of the pooling window as starting from the upper-left of the input tensor, and sliding across the input tensor from left to right and top to bottom
- at each location that the pooling window hits, it computes the maximum or average value of the input subtensor in the window, depending on whether max or average pooling is employed

3.5 Pooling

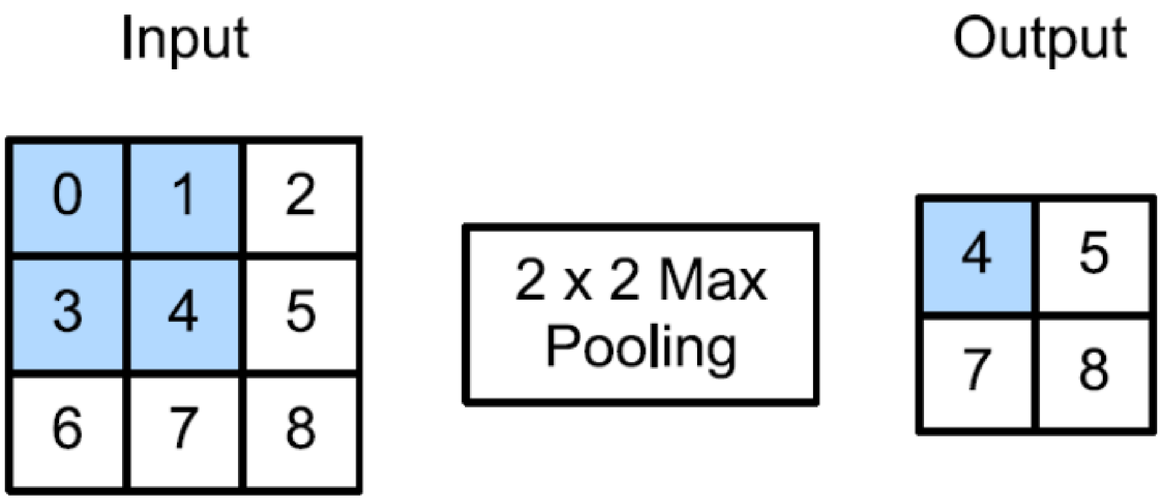

Figure 8: Maximum pooling with a pooling window shape of 2×2. The shaded portions are the first output element, as well as the input tensor elements used for the output computation: $\max(0, 1, 3, 4) = 4$.

3.5 Pooling

- the output tensor in Figure 8 has a height of 2 and a width of 2
- the four elements are derived from the maximum value in each pooling window:

$$\max(0, 1, 3, 4) = 4,$$
$$\max(1, 2, 4, 5) = 5,$$
$$\max(3, 4, 6, 7) = 7,$$
$$\max(4, 5, 7, 8) = 8.$$

- a pooling layer with a pooling window shape of $p \times q$ is called a $p \times q$ *pooling layer*
- the pooling operation is called $p \times q$ *pooling*

3.5 Pooling

- let us return to the object edge detection example mentioned at the beginning of this section
- now, we will use the output of the convolutional layer as the input for 2×2 maximum pooling
- set the convolutional layer input as $\mathbf{X}$ and the pooling layer output as $\mathbf{Y}$
- whether or not the values of $[\mathbf{X}]_{i,j}$ and $[\mathbf{X}]_{i,j+1}$ are different, or $[\mathbf{X}]_{i,j+1}$ and $[\mathbf{X}]_{i,j+2}$ are different, the pooling layer always outputs $[\mathbf{Y}]_{i,j} = 1$
- that is to say, using the 2×2 maximum pooling layer, we can still detect if the pattern recognized by the convolutional layer moves no more than one element in height or width

3.5 Pooling

- as with convolutional layers, pooling layers can also change the output shape
- and, as before, we can alter the operation to achieve a desired output shape by padding the input and adjusting the stride
- when processing multi-channel input data, the pooling layer pools each input channel separately, rather than summing the inputs up over channels, as in a convolutional layer
- this means that the number of output channels for the pooling layer is the same as the number of input channels

3.6 Convolutional neural networks (LeNet)

- we now have all the ingredients required to assemble a fully-functional CNN
- in this section, we will introduce *LeNet*, among the first published CNNs to capture wide attention for its performance on computer vision tasks
- the model was introduced by (and named for) Yann LeCun, then a researcher at AT&T Bell Labs, for the purpose of recognizing handwritten digits in images
- this work represented the culmination of a decade of research developing the technology
- in 1989, LeCun published the first study to successfully train CNNs via backpropagation

3.6 Convolutional neural networks (LeNet)

- at the time, LeNet achieved outstanding results, matching the performance of support vector machines, then a dominant approach in supervised learning
- LeNet was eventually adapted to recognize digits for processing deposits in ATM machines; to this day, some ATMs still run the code that Yann LeCun and his colleague Leon Bottou wrote in the 1990s
- at a high level, LeNet (LeNet-5) consists of two parts:
  - a convolutional encoder consisting of two convolutional layers
  - a dense block consisting of three fully-connected layers; the architecture is summarized in Figure 9

3.6 Convolutional neural networks (LeNet)

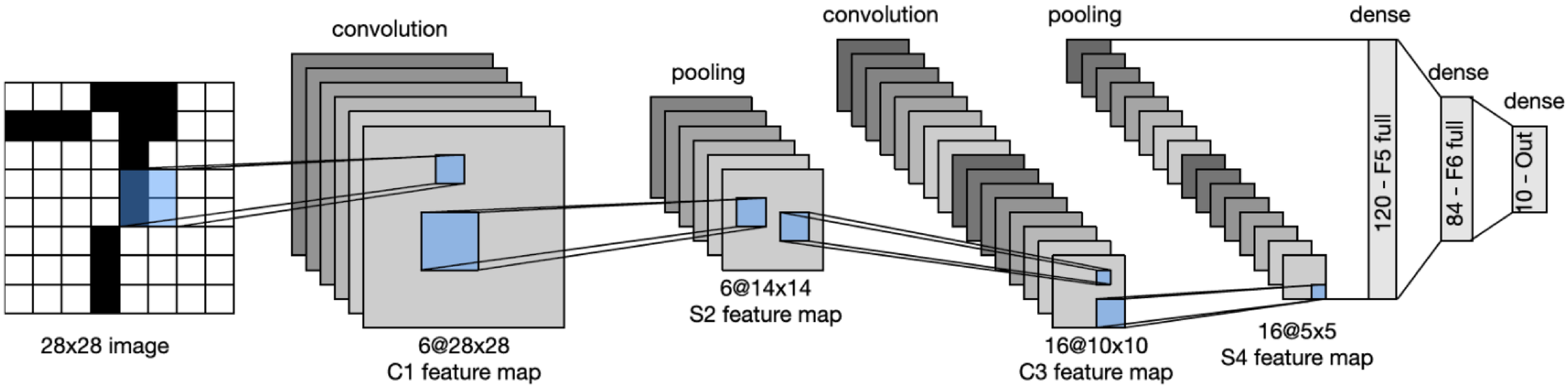

Figure 9: Data flow in LeNet. The input is a handwritten digit and the output is a probability over 10 possible outcomes.

3.6 Convolutional neural networks (LeNet)

- the basic units in each convolutional block are a convolutional layer, a sigmoid activation function, and a subsequent average pooling operation
- note that, while ReLUs and max-pooling work better, these discoveries had not yet been made in the 1990s
- each convolutional layer uses a 5×5 kernel and a sigmoid activation function
- these layers map spatially arranged inputs to a number of two-dimensional feature maps, typically increasing the number of channels
- the first convolutional layer has 6 output channels, while the second has 16
- each 2×2 pooling operation (stride 2) reduces dimensionality by a factor of 4 via spatial downsampling
- the convolutional block emits an output with shape given by (batch size, number of channels, height, width)

3.6 Convolutional neural networks (LeNet)

- in order to pass output from the convolutional block to the dense block, we must *flatten* each example in the mini-batch
- in other words, we take this four-dimensional input and transform it into the two-dimensional input expected by fully-connected layers: as a reminder, the two-dimensional representation that we desire uses the first dimension to index examples in the mini-batch and the second to give the flat vector representation of each example
- LeNet's dense block has three fully-connected layers, with 120, 84, and 10 outputs, respectively
- because we are still performing classification, the 10-dimensional output layer corresponds to the number of possible output classes
- by passing a single-channel (black and white) 28×28 image through the network, the model is depicted in Figure 10
- we took a small liberty with the original model, removing the Gaussian activation in the final layer; other than that, this network matches the original LeNet-5 architecture

3.6 Convolutional neural networks (LeNet)

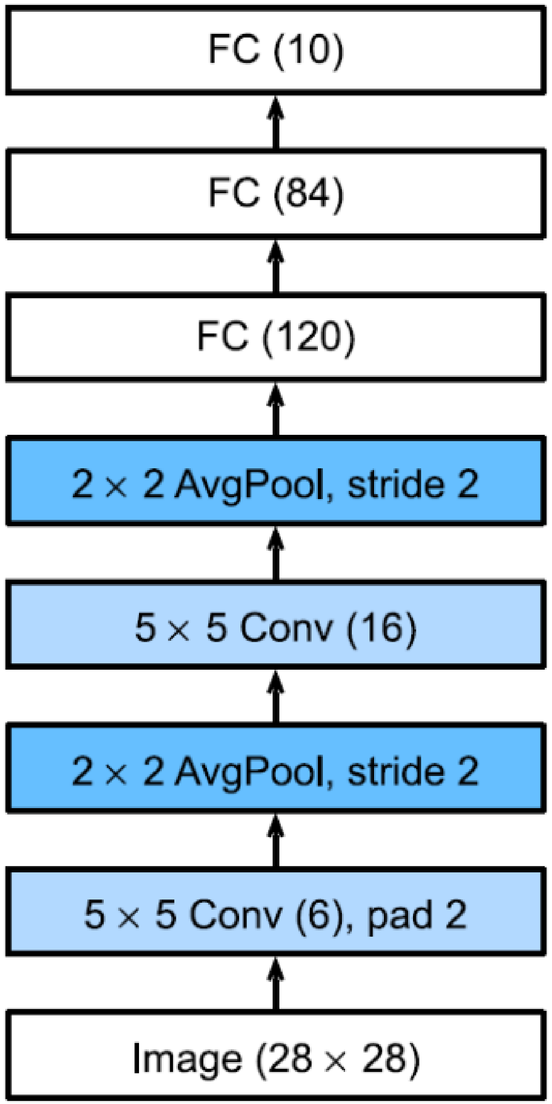

Figure 10: Compressed notation for LeNet-5.

3.6 Convolutional neural networks (LeNet)

- note that the height and width of the representation at each layer throughout the convolutional block is reduced (compared with the previous layer)
- the first convolutional layer uses 2 pixels of padding to compensate for the reduction in height and width that would otherwise result from using a 5×5 kernel
- in contrast, the second convolutional layer doesn't use padding, and thus the height and width are both reduced by 4 pixels
- as we go up the stack of layers, the number of channels increases layer-over-layer from 1 in the input to 6 after the first convolutional layer and 16 after the second convolutional layer
- however, each pooling layer halves the height and width
- finally, each fully-connected layer reduces dimensionality, finally emitting an output whose dimension matches the number of classes

3.7 Deep convolutional neural networks (AlexNet)

- now that we understand the basics of wiring together CNNs, we will take a tour of modern CNN architectures
- each next section will correspond to a significant CNN architecture that was briefly a dominant architecture and were winners (AlexNet, ResNet) or runners-up (VGG) in the ImageNet competition, which has served as a barometer of progress on supervised learning in computer vision since 2010
- these models include AlexNet, the first large-scale network deployed to beat conventional computer vision methods on a large-scale vision challenge; the VGG network, which makes use of a number of repeating blocks of elements; and residual networks (ResNet), which remain the most popular off-the-shelf architecture in computer vision

3.7 Deep convolutional neural networks (AlexNet)

- while the idea of *deep neural networks* is quite simple (stack together a bunch of layers), performance can vary wildly across architectures and hyperparameter choices
- the neural networks described next are the product of intuition, a few mathematical insights, and a whole lot of trial and error
- we present these models in chronological order, partly to convey a sense of the history, so that we can form our own intuitions about where the field is heading, and perhaps develop our own architectures
- for instance, batch normalization and residual connections, described next, have offered two popular ideas for training and designing deep models

3.7 Deep convolutional neural networks (AlexNet)

- although CNNs were well known in the computer vision and machine learning communities following the introduction of LeNet, they did not immediately dominate the field
- although LeNet achieved good results on early small datasets, the performance and feasibility of training CNNs on larger, more realistic datasets had yet to be established
- in fact, for much of the intervening time between the early 1990s and the breakthrough results of 2012, neural networks were often surpassed by other machine learning methods, such as support vector machines

3.7 Deep convolutional neural networks (AlexNet)

- for computer vision, this comparison is perhaps not fair
- that is, although the inputs to convolutional networks consist of raw or lightly-processed (e.g., by centering) pixel values, practitioners would never feed raw pixels into traditional models
- instead, typical computer vision pipelines consisted of *manually* engineering *feature extraction* pipelines
- rather than *learn the features*, the features were *crafted*
- most of the progress came from having more clever ideas for features, and the learning algorithm was often very simple

3.7 Deep convolutional neural networks (AlexNet)

- although some neural network accelerators were available in the 1990s, they were not yet sufficiently powerful to make deep multichannel, multilayer CNNs with a large number of parameters
- moreover, datasets were still relatively small
- added to these obstacles, key tricks for training neural networks including parameter initialization heuristics, clever variants of stochastic gradient descent, non-squashing activation functions, and effective regularization techniques were still missing
- thus, rather than training *end-to-end* (pixel to classification) systems, classical pipelines looked more like this:
  1. Obtain an interesting dataset. In early days, these datasets required expensive sensors (at the time, 1 megapixel images were state-of-the-art).
  2. Preprocess the dataset with hand-crafted features based on some knowledge of optics, geometry, other analytic tools, and occasionally on the discoveries of lucky graduate students.
  3. Feed the data through a standard set of feature extractors such as the SIFT (scale-invariant feature transform), the SURF (speeded up robust features), or any number of other hand-tuned pipelines.
  4. Dump the resulting representations into a classifier, likely a linear model or kernel method, to train a classifier.

3.7 Deep convolutional neural networks (AlexNet)

- another group of researchers, including Yann LeCun, Geoffrey Hinton, Yoshua Bengio, Andrew Ng, and Juergen Schmidhuber had different plans: they believed that features themselves should be learned
- moreover, they believed that, to be reasonably complex, the features should be *hierarchically* composed with multiple jointly learned layers, each with learnable parameters
- in the case of an image, the lowest layers might come to detect edges, colors, and textures
- indeed, Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton proposed a new variant of a CNN, *AlexNet*, that achieved excellent performance in the 2012 ImageNet challenge
- AlexNet was named after Alex Krizhevsky, the first author of the breakthrough ImageNet classification paper

3.7 Deep convolutional neural networks (AlexNet)

- interestingly, in the lowest layers of the network, the model learned feature extractors that resembled some traditional filters; Figure 11 is reproduced from the AlexNet paper, and describes lower-level image descriptors

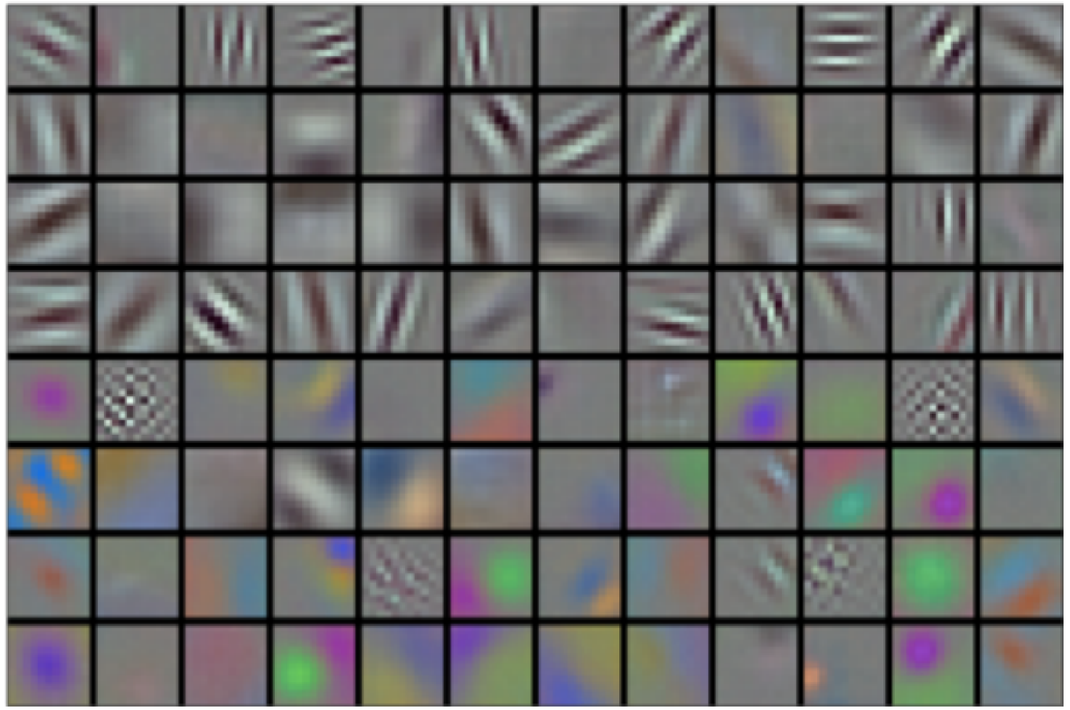

Figure 11: Image filters learned by the first layer of AlexNet.

3.7 Deep convolutional neural networks (AlexNet)

- higher layers in the network might build upon these representations to represent larger structures, like eyes, noses, blades of grass, and so on
- even higher layers might represent whole objects like people, airplanes, dogs, or frisbees
- ultimately, the final hidden state learns a compact representation of the image that summarizes its contents, such that data belonging to different categories can be easily separated
- while the ultimate breakthrough for many-layered CNNs came in 2012, a core group of researchers had dedicated themselves to this idea, attempting to learn *hierarchical representations* of visual data for many years
- the ultimate breakthrough in 2012 can be attributed to *two key factors*

3.7 Deep convolutional neural networks (AlexNet)

- deep models with many layers require large amounts of *data* in order to enter the regime where they significantly outperform traditional methods based on convex optimizations (e.g., linear and kernel methods)
- however, given the limited storage capacity of computers, the relative expense of sensors, and the comparatively tighter research budgets in the 1990s, most research relied on tiny datasets
- numerous papers addressed the UCI collection of datasets, many of which contained only hundreds or (a few) thousands of images captured in unnatural settings with low resolution

3.7 Deep convolutional neural networks (AlexNet)

- in 2009, the ImageNet dataset was released, challenging researchers to learn models from 1 million examples, each from one of 1000 distinct categories of objects
- the researchers, led by Fei-Fei Li, who introduced this dataset, leveraged Google Image Search to prefilter large candidate sets for each category and employed the Amazon Mechanical Turk crowdsourcing pipeline to confirm for each image whether it belonged to the associated category
- this scale was unprecedented
- the associated competition, named the ImageNet Challenge pushed computer vision and machine learning research forward, challenging researchers to identify which models performed best at a greater scale than academics had previously considered

3.7 Deep convolutional neural networks (AlexNet)

- the second factor is *hardware*
- deep learning models are massive consumers of compute cycles
- training can take hundreds of epochs, and each iteration requires passing data through many layers of computationally-expensive linear algebra operations
- this is one of the main reasons why, in the 1990s and early 2000s, simple algorithms based on the more-efficiently optimized convex objectives were preferred

3.7 Deep convolutional neural networks (AlexNet)

- *graphical processing units* (GPUs) proved to be a game changer in making deep learning feasible
- these chips had long been developed for accelerating graphics processing to benefit computer games
- in particular, they were optimized for high throughput 4×4 matrix-vector products, which are needed for many computer graphics tasks
- fortunately, this math is strikingly similar to that required to calculate convolutional layers
- around that time, NVIDIA and ATI had begun optimizing GPUs for general computing operations, going as far as to market them as *general-purpose GPUs* (GPGPU)

3.7 Deep convolutional neural networks (AlexNet)

- back to 2012; a major breakthrough came when Alex Krizhevsky and Ilya Sutskever implemented a deep CNN that could run on GPU hardware
- they realized that the computational bottlenecks in CNNs, convolutions and matrix multiplications, are all operations that could be parallelized in hardware
- using two NVIDIA GTX 580s with 3GB of memory, they implemented fast convolutions
- the code `cuda-convnet` was good enough that, for several years, it was the industry standard, and powered the first couple years of the deep learning boom

3.7 Deep convolutional neural networks (AlexNet)

- AlexNet, which employed an 8-layer CNN, won the ImageNet Large Scale Visual Recognition Challenge 2012 by a phenomenally large margin
- this network showed, for the first time, that the features obtained by learning can transcend manually-designed features, breaking the previous paradigm in computer vision
- the architectures of AlexNet and LeNet are very similar, as Figure 12 illustrates
- note that we provide a slightly streamlined version of AlexNet, removing some of the design tricks that were needed in 2012 to make the model fit on two small GPUs

3.7 Deep convolutional neural networks (AlexNet)

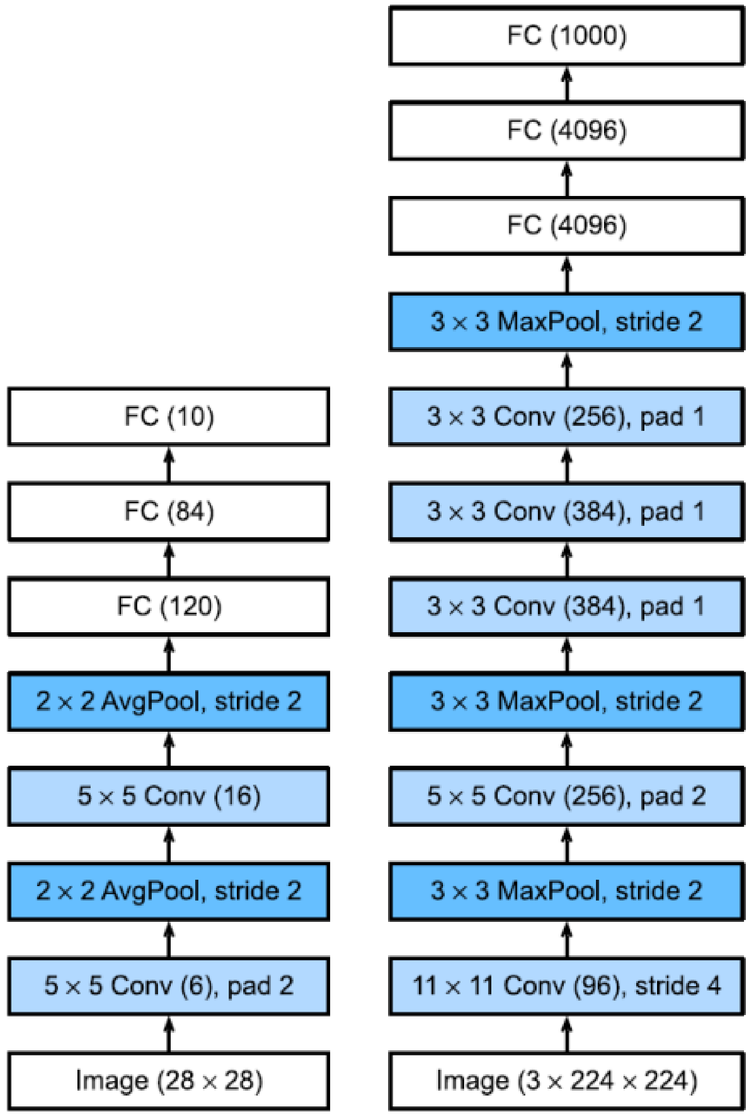

Figure 12: From LeNet (left) to AlexNet (right).

3.7 Deep convolutional neural networks (AlexNet)

- the design philosophies of AlexNet and LeNet are very similar, but there are also significant differences
- first, AlexNet is much deeper than the comparatively small LeNet5
- AlexNet consists of eight layers: five convolutional layers, two fully-connected hidden layers, and one fully-connected output layer
- second, AlexNet used the ReLU instead of the sigmoid as its activation function

3.7 Deep convolutional neural networks (AlexNet)

- in AlexNet's first layer, the convolution window shape is 11×11
- since most images in ImageNet are more than ten times higher and wider than the MNIST images, objects in ImageNet data tend to occupy more pixels
- consequently, a larger convolution window is needed to capture the object
- the convolution window shape in the second layer is reduced to 5×5, followed by 3×3
- in addition, after the first, second, and fifth convolutional layers, the network adds maximum pooling layers with a window shape of 3×3 and a stride of 2
- moreover, AlexNet has ten times more convolution channels than LeNet

3.7 Deep convolutional neural networks (AlexNet)

- after the last convolutional layer there are two fully-connected layers with 4096 outputs
- these two huge fully-connected layers produce model parameters of nearly 1 GB
- due to the limited memory in early GPUs, the original AlexNet used a dual data stream design, so that each of their two GPUs could be responsible for storing and computing only its half of the model
- fortunately, GPU memory is comparatively abundant now, so we rarely need to break up models across GPUs these days (our version of the AlexNet model deviates from the original paper in this aspect)

3.7 Deep convolutional neural networks (AlexNet)

- besides, AlexNet changed the sigmoid activation function to a simpler ReLU activation function
- on one hand, the computation of the ReLU activation function is simpler; for example, it does not have the exponentiation operation found in the sigmoid activation function
- on the other hand, the ReLU activation function makes model training easier when using different parameter initialization methods
- this is because, when the output of the sigmoid activation function is very close to 0 or 1, the gradient of these regions is almost 0, so that backpropagation cannot continue to update some of the model parameters
- in contrast, the gradient of the ReLU activation function in the positive interval is always 1
- therefore, if the model parameters are not properly initialized, the sigmoid function may obtain a gradient of almost 0 in the positive interval, so that the model cannot be effectively trained

3.7 Deep convolutional neural networks (AlexNet)

- AlexNet controls the model complexity of the fully-connected layer by dropout (Section 2.5), while LeNet only uses weight decay
- to augment the data even further, the training loop of AlexNet added a great deal of image augmentation, such as flipping, clipping, and color changes
- this makes the model more robust, and the larger sample size effectively reduces overfitting

3.8 Networks using blocks (VGG)

- while AlexNet offered empirical evidence that deep CNNs can achieve good results, it did not provide a general template to guide subsequent researchers in designing new networks
- in the following sections, we will introduce several heuristic concepts commonly used to design deep networks
- progress in this field mirrors that in chip design, where engineers went from placing transistors to logical elements to logic blocks
- similarly, the design of neural network architectures had grown progressively more abstract, with researchers moving from thinking in terms of individual neurons to whole layers, and now to blocks, repeating patterns of layers
- the idea of using blocks first emerged from the Visual Geometry Group (VGG) at Oxford University, which gave the name of their *VGG* network
- it is easy to implement these repeated structures in code with any modern deep learning framework, by using loops and subroutines

3.8 Networks using blocks (VGG)

- the basic building block of classic CNNs is a sequence of the following:
  - a convolutional layer with padding to maintain the resolution
  - a nonlinearity such as a ReLU
  - a pooling layer such as a maximum pooling layer
- one VGG block consists of a sequence of convolutional layers, followed by a maximum pooling layer for spatial downsampling
- in the original VGG paper, the authors employed convolutions with 3×3 kernels with padding of 1 (keeping height and width) and 2×2 maximum pooling with stride of 2 (halving the resolution after each block)
- like AlexNet and LeNet, the VGG Network can be partitioned into two parts: the first consisting mostly of convolutional and pooling layers, and the second consisting of fully-connected layers; this is depicted in Figure 13

3.8 Networks using blocks (VGG)

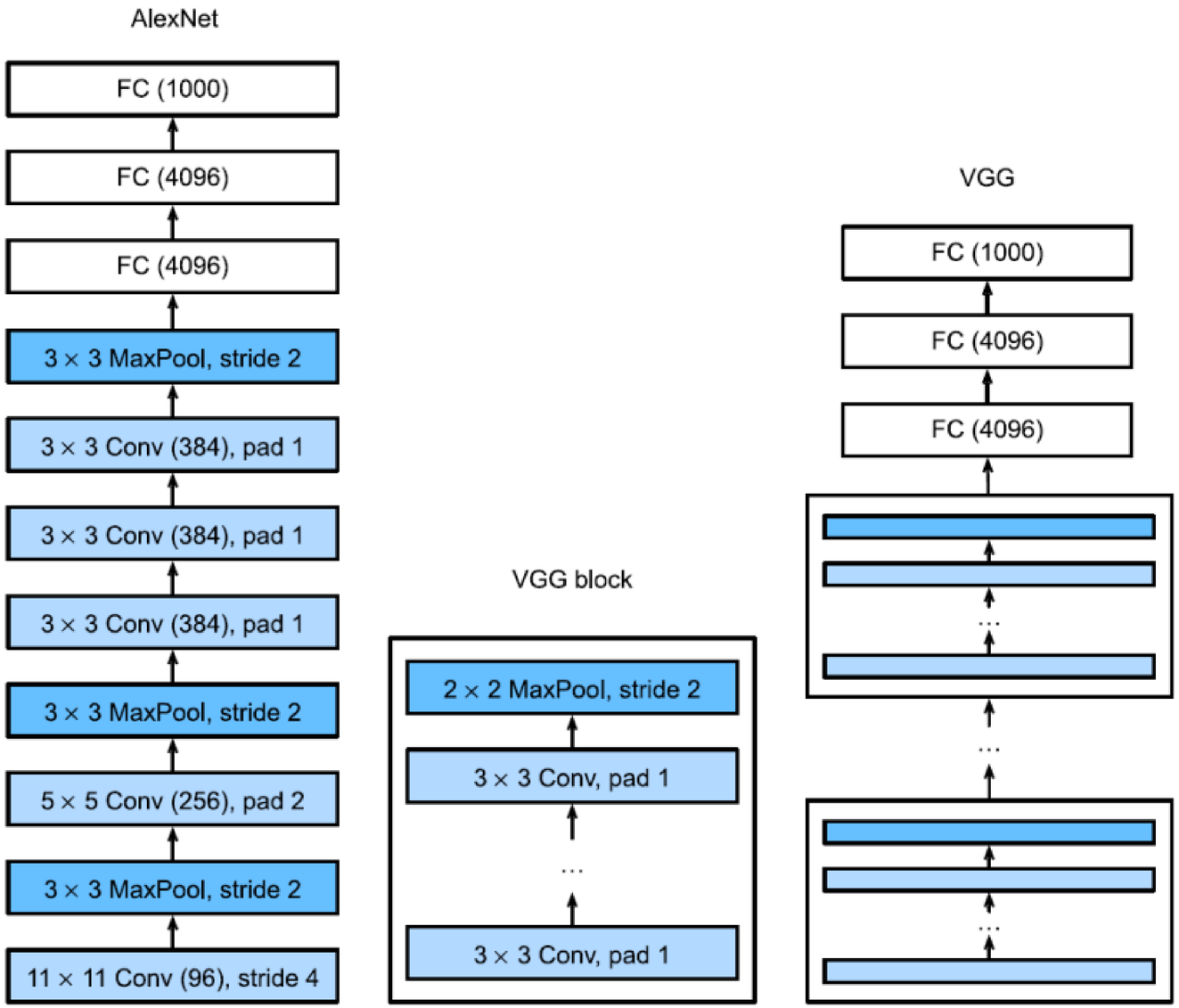

Figure 13: From AlexNet to VGG, which is designed from building blocks.

3.8 Networks using blocks (VGG)

- the convolutional part of the network connects several VGG blocks from Figure 13, in succession
- the fully-connected part of the VGG network is identical to that covered in AlexNet
- the original VGG network had 5 convolutional blocks, among which the first two have one convolutional layer each, and the latter three contain two convolutional layers each
- the first block has 64 output channels, and each subsequent block doubles the number of output channels, until that number reaches 512
- since this network uses 8 convolutional layers and 3 fully-connected layers, it is often called *VGG-11*

3.9 Batch normalization

- training deep neural networks is difficult, and getting them to converge in a reasonable amount of time can be tricky
- in this section, we describe *batch normalization*, a popular and effective technique that consistently accelerates the convergence of deep networks
- together with residual blocks – covered later, in Section 3.10 – batch normalization has made it possible to routinely train networks with over 100 layers
- to motivate batch normalization, let us review a few practical challenges that arise when training machine learning models, and neural networks in particular

3.9 Batch normalization

- first, choices regarding data preprocessing often make an enormous difference in the final results
- our first step when working with data is to standardize our input features to each have a mean of zero and variance of one
- intuitively, this standardization plays nicely with our optimizers, because it puts the parameters *a priori* at a similar scale
- second, for a typical MLP or CNN, as we train, the variables (e.g., affine transformation outputs in MLP) in intermediate layers may take values with widely varying magnitudes: both along the layers from the input to the output, across units in the same layer, and over time due to our updates to the model parameters

3.9 Batch normalization

- the inventors of batch normalization postulated informally that this drift in the distribution of such variables could prevent the convergence of the network
- intuitively, we might conjecture that, if one layer has variable values that are 100 times that of another layer, this might need compensatory adjustments in the learning rates
- third, deeper networks are complex and easily capable of overfitting; this means that regularization becomes more critical

3.9 Batch normalization

- batch normalization is applied to individual layers (optionally, to all of them) and works as follows: in each training iteration, we first normalize the inputs (of batch normalization) by subtracting their mean and dividing by their standard deviation, where both are estimated based on the statistics of the current mini-batch
- next, we apply a scale coefficient and a scale offset
- it is precisely due to this *normalization* based on *batch* statistics that *batch normalization* derives its name

3.9 Batch normalization

- note that, if we tried to apply batch normalization with mini-batches of size 1, we would not be able to learn anything
- that is because, after subtracting the means, each hidden unit would take value 0
- as we might guess, since we are devoting a whole section to batch normalization, with large enough mini-batches, the approach proves effective and stable
- one takeaway here is that, when applying batch normalization, the choice of batch size may be even more significant than without batch normalization

3.9 Batch normalization

- formally, denoting by $\mathbf{x} \in \mathcal{B}$ an input to batch normalization (BN) that is from a mini-batch $\mathcal{B}$, batch normalization transforms $\mathbf{x}$ according to the following expression:

$$\text{BN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \hat{\mu}_{\mathcal{B}}}{\hat{\sigma}_{\mathcal{B}}} + \beta. \tag{4}$$

- in (4), $\hat{\mu}_{\mathcal{B}}$ is the sample mean and $\hat{\sigma}_{\mathcal{B}}$ is the sample standard deviation of the mini-batch $\mathcal{B}$
- after applying standardization, the resulting mini-batch has zero mean and unit variance
- because the choice of unit variance (vs. some other number) is an arbitrary choice, we commonly include element-wise *scale parameter* γ and *shift parameter* β, which have the same shape as $\mathbf{x}$
- note that γ and β are parameters that need to be learned jointly with the other model parameters

3.9 Batch normalization

- consequently, the variable magnitudes for intermediate layers cannot diverge during training, because batch normalization actively centers and rescales them back to a given mean and size (via $\hat{\mu}_{\mathcal{B}}$ and $\hat{\sigma}_{\mathcal{B}}$)
- one piece of practitioner's intuition or wisdom is that batch normalization seems to allow for more aggressive learning rates
- formally, we calculate $\hat{\mu}_{\mathcal{B}}$ and $\hat{\sigma}_{\mathcal{B}}$ in (4) as follows:

$$\begin{aligned} \hat{\mu}_{\mathcal{B}} &= \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} \mathbf{x} \\ \hat{\sigma}^2_{\mathcal{B}} &= \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} (\mathbf{x} - \hat{\mu}_{\mathcal{B}})^2 + \epsilon. \end{aligned}$$

- note that we add a small constant $\epsilon > 0$ to the variance estimate to ensure that we never attempt division by zero, even in cases where the empirical variance estimate might vanish
- the estimates $\hat{\mu}_{\mathcal{B}}$ and $\hat{\sigma}_{\mathcal{B}}$ counteract the scaling issue by using noisy estimates of mean and variance
- we might think that this noisiness should be a problem; as it turns out, this is actually beneficial

3.9 Batch normalization

- fixing a trained model, we might think that we would prefer using the entire dataset to estimate the mean and variance
- once training is complete, why would we want the same image to be classified differently, depending on the batch in which it happens to reside?
- during training, such exact calculation is infeasible, because the intermediate variables for all data examples change every time we update our model
- however, once the model is trained, we can calculate the means and variances of each layer's variables based on the entire dataset
- indeed, this is standard practice for models employing batch normalization, and thus batch normalization layers function differently in *training mode* (normalizing by mini-batch statistics) and in *prediction mode* (normalizing by dataset statistics)

3.9 Batch normalization

- we are now ready to take a look at how batch normalization works in practice
- batch normalization implementations for fully-connected layers and convolutional layers are slightly different; we discuss both cases
- recall that one key difference between batch normalization and other layers is that, because batch normalization operates on a full mini-batch at a time, we cannot just ignore the batch dimension, as we did before, when introducing other layers

3.9 Batch normalization

- when applying batch normalization to fully-connected layers, the original paper inserts batch normalization after the affine transformation and before the nonlinear activation function (later applications may insert batch normalization right after activation functions)
- denoting the input to the fully-connected layer by $\mathbf{x}$, the affine transformation by $\mathbf{Wx} + \mathbf{b}$ (with the weight parameter $\mathbf{W}$ and the bias parameter $\mathbf{b}$), and the activation function by ϕ, we can express the computation of a batch-normalization-enabled, fully-connected layer output $\mathbf{h}$ as follows:

$$\mathbf{h} = \phi(\mathrm{BN}(\mathbf{Wx} + \mathbf{b})).$$

- recall that mean and variance are computed on the *same* mini-batch on which the transformation is applied

3.9 Batch normalization

- similarly, with convolutional layers, we can apply batch normalization after the convolution and before the nonlinear activation function
- when the convolution has multiple output channels, we need to carry out batch normalization for *each* of the outputs of these channels, and each channel has its own scale and shift parameters, both of which are scalars
- assume that our mini-batches contain m examples, and that, for each channel, the output of the convolution has height p and width q
- for convolutional layers, we carry out each batch normalization over the $m \cdot p \cdot q$ elements per output channel, simultaneously
- thus, we collect the values over all spatial locations when computing the mean and variance, and consequently apply the same mean and variance within a given channel to normalize the value at each spatial location

3.9 Batch normalization

- as we mentioned earlier, batch normalization typically behaves differently in training mode and prediction mode
- first, the noise in the sample mean and the sample variance arising from estimating each on mini-batches are no longer desirable, once we have trained the model
- second, we might not have the luxury of computing per-batch normalization statistics
- for example, we might need to apply our model to make one prediction at a time
- typically, after training, we use the entire dataset to compute stable estimates of the variable statistics, and then fix them at prediction time
- consequently, batch normalization behaves differently during training and at test time; recall that dropout also exhibits this characteristic

3.10 Residual networks (ResNet)

- as we design increasingly deeper networks, it becomes imperative to understand how adding layers can increase the complexity and expressiveness of the network
- even more important is the ability to design networks where adding layers makes networks strictly more expressive, rather than just different
- this is the question that He et al. considered when working on very deep computer vision models
- at the heart of their proposed *residual network* (*ResNet*) is the idea that every additional layer should more easily contain the *identity function* as one of its elements
- these considerations are rather profound, but they led to a surprisingly simple solution, the *residual block*
- with it, ResNet won the ImageNet Large Scale Visual Recognition Challenge in 2015
- the design had a profound influence on how to build deep neural networks

3.10 Residual networks (ResNet)

- let us focus on a local part of a neural network, as depicted in Figure 14
- denote the input by $\mathbf{x}$, and assume that the desired underlying mapping we want to obtain by learning is $f(\mathbf{x})$, to be used as the input to the activation function on the top
- on the left of Figure 14, the portion within the dotted-line box must directly learn the mapping $f(\mathbf{x})$
- on the right, the portion within the dotted-line box needs to learn the *residual mapping* $f(\mathbf{x}) - \mathbf{x}$, which is how the residual block derives its name
- if the identity mapping $f(\mathbf{x}) = \mathbf{x}$ is the desired underlying mapping, the residual mapping is easier to learn: we only need to push the weights and biases of the upper weight layer (e.g., fully-connected layer and convolutional layer) within the dotted-line box to zero
- the right figure in Figure 14 illustrates the *residual block* of ResNet, where the solid line carrying the layer input $\mathbf{x}$ to the addition operator is called a *residual connection* (or *shortcut connection*)
- with residual blocks, inputs can forward propagate faster through the residual connections across layers

3.10 Residual networks (ResNet)

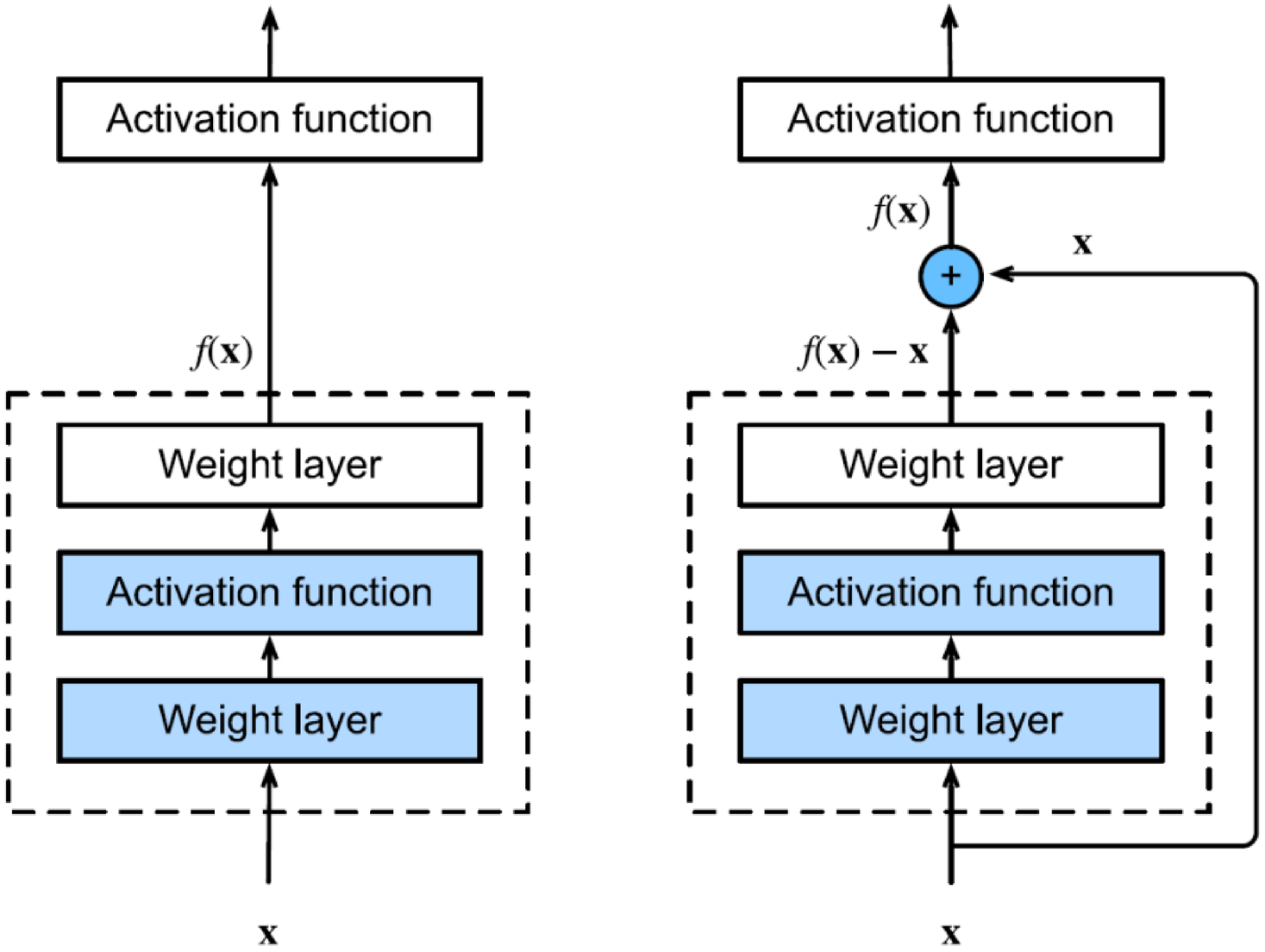

Figure 14: A regular block (left) and a residual block (right).

3.10 Residual networks (ResNet)

- ResNet follows VGG's full 3×3 convolutional layer design
- the residual block has two 3×3 convolutional layers with the same number of output channels
- each convolutional layer is followed by a batch normalization layer and a ReLU activation function
- then, we skip these two convolution operations and add the input directly before the final ReLU activation function
- this kind of design requires that the output of the two convolutional layers has to be of the same shape as the input, so that they can be added together
- if we want to change the number of channels or the resolution, we need to introduce an additional 1×1 convolutional layer to transform the input into the desired shape for the addition operation
- thus, we have two types of blocks: one where we add the input to the output before applying the ReLU nonlinearity, and one where we adjust channels and resolution by means of a 1×1 convolution before adding; Figure 15 illustrates this

3.10 Residual networks (ResNet)

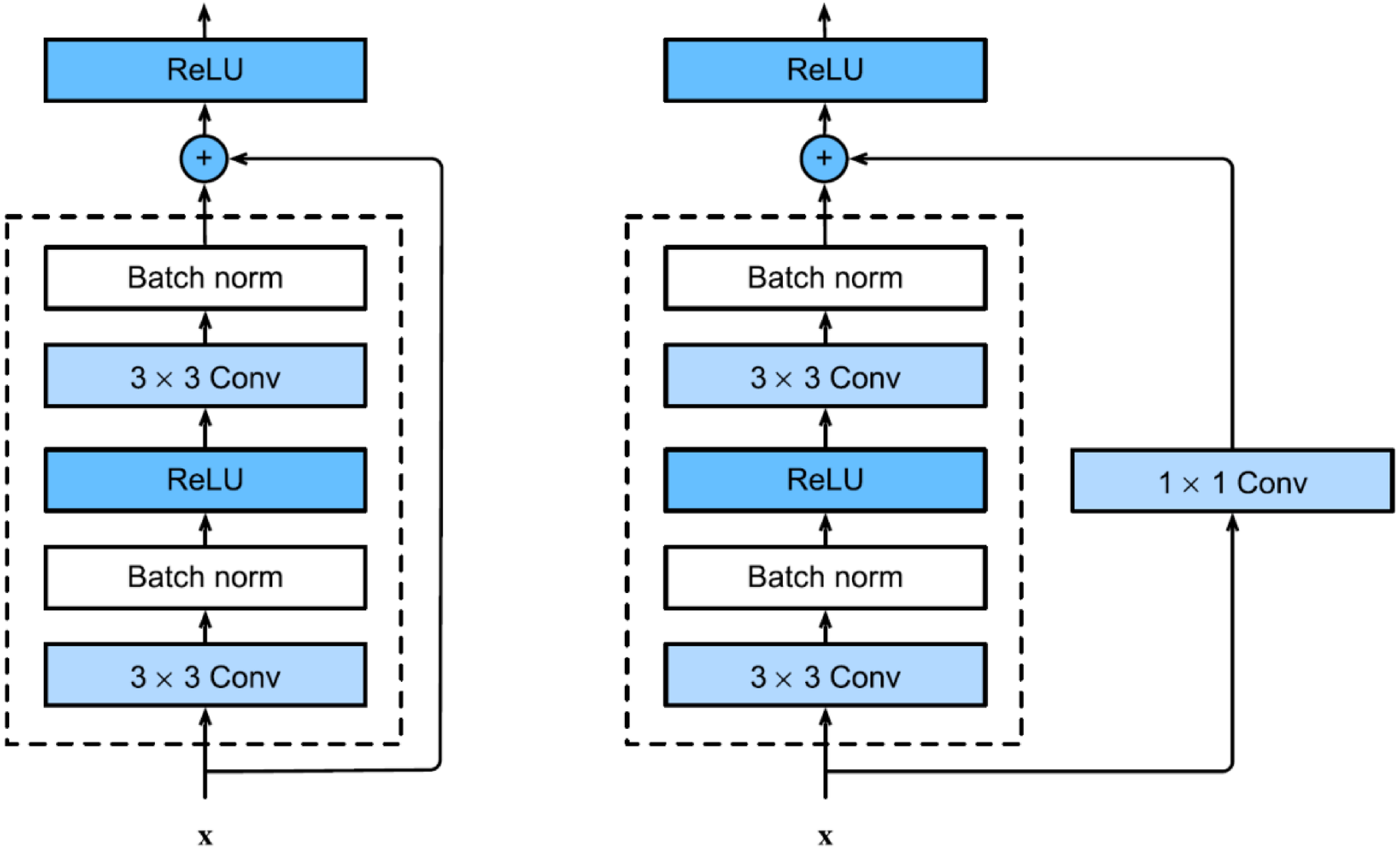

Figure 15: ResNet block with and without 1×1 convolution.

3.10 Residual networks (ResNet)

- the first two layers of ResNet are a 7×7 convolutional layer with 64 output channels and a stride of 2, which is followed by the 3×3 maximum pooling layer with a stride of 2
- a batch normalization layer is added after each convolutional layer in ResNet
- ResNet uses four modules made up of residual blocks, each of which uses several residual blocks with the same number of output channels
- the number of channels in the first module is the same as the number of input channels
- since a maximum pooling layer with a stride of 2 has already been used, it is not necessary to reduce the height and width
- in the first residual block, for each of the subsequent modules, the number of channels is doubled compared with that of the previous module, and the height and width are halved

3.10 Residual networks (ResNet)

- finally, we add a global average pooling layer, followed by the fully-connected layer output
- there are 4 convolutional layers in each module (excluding the 1×1 convolutional layer)
- together with the first 7×7 convolutional layer and the final fully-connected layer, there are 18 layers in total
- therefore, this model is commonly known as *ResNet-18*
- by configuring different numbers of channels and residual blocks in the module, we can create different ResNet models, such as the deeper 152-layer *ResNet-152*
- the main architecture of ResNet is simple and easy to modify, which has resulted in the rapid and widespread use of ResNet; Figure 16 depicts the full ResNet-18

3.10 Residual networks (ResNet)

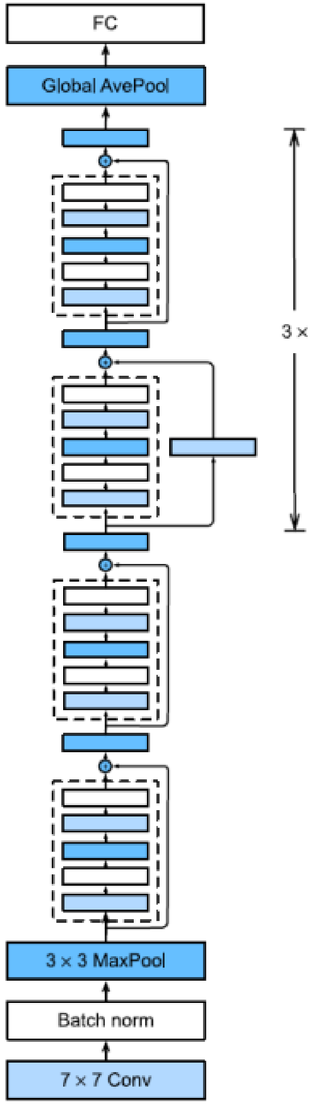

Figure 16: The ResNet-18 architecture.

Exercise 3

Define a convolutional layer with 3 input channels, 2 output channels, 2×2 kernel. Define a 3×3 input for each input channel, the kernel, and the bias, and compute the output of the layer. Specify the total number of real parameters (weights and biases) for the layer.
Solution:

- we have the following formula:

$$[\mathsf{H}]_{i,j,d} = [\mathbf{u}]_d + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} \sum_{c} [\mathsf{V}]_{a,b,c,d} [\mathsf{X}]_{i+a,j+b,c},$$

where $\mathbf{u} \in \mathbb{R}^{c_{out} \times 1}$, $\mathsf{X} \in \mathbb{R}^{c_{in} \times n_h \times n_w}$, $\mathsf{V} \in \mathbb{R}^{c_{out} \times c_{in} \times k_h \times k_w}$, $\mathsf{H} \in \mathbb{R}^{c_{out} \times n'_h \times n'_w}$, and

$$n'_h \times n'_w = (n_h - k_h + 1) \times (n_w - k_w + 1).$$

Exercises

- in our case, $n_h = n_w = 3$, $c_{in} = 3$, $c_{out} = 2$, $k_h = k_w = 2$, and

$$X = \begin{bmatrix} \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}, \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \begin{bmatrix} 2 & 3 & 4 \\ 5 & 6 & 7 \\ 8 & 9 & 10 \end{bmatrix} \end{bmatrix} \in \mathbb{R}^{3\times3\times3},$$

$$V = \begin{bmatrix} \begin{bmatrix} \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}, \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} \end{bmatrix}, \begin{bmatrix} \begin{bmatrix} 3 & 4 \\ 5 & 6 \end{bmatrix}, \begin{bmatrix} 4 & 5 \\ 6 & 7 \end{bmatrix}, \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \end{bmatrix} \end{bmatrix} \in \mathbb{R}^{2\times3\times2\times2},$$

$$\mathbf{u} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \in \mathbb{R}^{2\times1}.$$

Exercises

- now, we apply the formula:

$$
\begin{aligned}
[H]_{\cdot,\cdot,1} &= \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix} * \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} + \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} * \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \\
&\quad + \begin{bmatrix} 2 & 3 & 4 \\ 5 & 6 & 7 \\ 8 & 9 & 10 \end{bmatrix} * \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} + [1] \\
&= \begin{bmatrix} 19 & 25 \\ 37 & 43 \end{bmatrix} + \begin{bmatrix} 37 & 47 \\ 67 & 77 \end{bmatrix} + \begin{bmatrix} 63 & 77 \\ 105 & 119 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \\
&= \begin{bmatrix} 120 & 150 \\ 210 & 240 \end{bmatrix} \in \mathbb{R}^{2\times 2},
\end{aligned}
$$

Exercises

$$
\begin{aligned}
[H]_{.,.,2} &= \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix} * \begin{bmatrix} 3 & 4 \\ 5 & 6 \end{bmatrix} + \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} * \begin{bmatrix} 4 & 5 \\ 6 & 7 \end{bmatrix} \\
&\quad + \begin{bmatrix} 2 & 3 & 4 \\ 5 & 6 & 7 \\ 8 & 9 & 10 \end{bmatrix} * \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} + [2] \\
&= \begin{bmatrix} 43 & 61 \\ 97 & 115 \end{bmatrix} + \begin{bmatrix} 73 & 95 \\ 139 & 161 \end{bmatrix} + \begin{bmatrix} 111 & 137 \\ 189 & 215 \end{bmatrix} + \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} \\
&= \begin{bmatrix} 229 & 295 \\ 427 & 493 \end{bmatrix} \in \mathbb{R}^{2\times 2},
\end{aligned}
$$

Exercises

$$H = \left[\begin{bmatrix} 120 & 150 \\ 210 & 240 \end{bmatrix}, \begin{bmatrix} 229 & 295 \\ 427 & 493 \end{bmatrix} \right] \in \mathbb{R}^{2\times 2\times 2},$$

$$\begin{aligned} n'_h \times n'_w &= (3-2+1)\times(3-2+1) \\ &= 2\times 2, \end{aligned}$$

$$\begin{aligned} \text{Total parameters} &= 2\times 3\times 2\times 2 + 2\times 1 \\ &= 24+2 \\ &= 26. \end{aligned}$$

Vă mulţumesc!